



UNIVERSIDAD DE MÁLAGA



GRADO EN INGENIERÍA DE COMPUTADORES

Fail Tolerant DualNano – Arquitectura Redundante para el Control de Sistemas Críticos Fail Tolerant DualNano – Redundant Architecture for Control of Critical Systems

Realizado por
Rafael Ramírez Salas

Tutorizado por
Pablo Pérez Trabado

Departamento
Arquitectura de Computadores

MÁLAGA, JUNIO 2024



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADO EN INGENIERÍA DE COMPUTADORES

Fail Tolerant DualNano - Arquitectura Redundante para el Control de Sistemas Críticos

Fail Tolerant DualNano – Redundant Architecture for Control of Critical Systems

Realizado por
Rafael Ramírez Salas

Tutorizado por
Pablo Pérez Trabado

Departamento
Arquitectura de Computadores

UNIVERSIDAD DE MÁLAGA
MÁLAGA, JUNIO DE 2024

Fecha defensa: julio de 2024

Resumen

En entornos industriales es mandatorio el uso de redundancia en los mecanismos de control de sistemas críticos (como, por ejemplo, los tanques de reacción), de forma que un fallo del hardware de control no se traduzca en interrupciones del proceso, o incluso en un accidente. Dado que este control se suele realizar usando microcontroladores, se hace necesario usar una arquitectura que permita la operación concurrente y cooperativa de dos o más microcontroladores para proporcionar la necesaria redundancia.

En este proyecto se ha diseñado y realizado un ejemplo de arquitectura real de control con redundancia activa, mediante duplicación, para el control de un proceso industrial (representado por una vasija de control simulada). La arquitectura implementada utiliza tres microcontroladores Arduino Nano, de los cuales dos realizan en paralelo el control del proceso, procesando los datos de los sensores y enviando comandos a los actuadores, mientras el tercer Arduino actúa como comparador y coordinador de las respuestas de los otros dos, detectando la aparición de discrepancias en estas respuestas, y tomando acciones correctoras si es necesario.

El desarrollo de este proyecto ha requerido un esfuerzo multidisciplinar, puesto que ha involucrado el diseño de la arquitectura hardware de comunicaciones y redundancia, su implementación física usando Arduinos reales, el diseño y programación del algoritmo de control, y el diseño y programación de los interfaces de usuario, así como el empleo de los diversos kits de desarrollo necesarios para la programación de los Arduinos (en C++) y de las interfaces gráficas (en C++ y Python). El proyecto, por tanto, permite demostrar ampliamente el nivel de suficiencia adquirido en el uso de las competencias aprendidas en la titulación del Grado Ingeniería de Computadores.

Palabras Clave: Arduino, arquitectura redundante, microcontrolador, redundancia activa por comparación, sistema de control

Abstract

In industrial environments, it is mandatory to use redundancy in the control mechanisms of critical systems (e.g. reaction tanks), so that a failure of the control hardware does not result in process interruptions or even an accident. Since this control is usually performed using microcontrollers, it is necessary to use an architecture that allows the concurrent and cooperative operation of two or more microcontrollers to provide the necessary redundancy.

In this project, an example of a real control architecture with active redundancy, through duplication, has been designed and realised for the control of an industrial process (represented by a simulated control vessel). The implemented architecture uses three Arduino Nano microcontrollers, two of which control the process in parallel, processing the data from the sensors and sending commands to the actuators, while the third Arduino acts as a comparator and coordinator of the responses of the other two, detecting the appearance of discrepancies in these responses, and taking corrective action if necessary.

The development of this project has required a multidisciplinary effort, as it has involved the design of the hardware architecture for communications and redundancy, its physical implementation using real Arduinos, the design and programming of the control algorithm, and the design and programming of the user interfaces, as well as the use of the various development kits necessary for programming the Arduinos (in C++) and the graphic interfaces (in C++ and Python). The project, therefore, allows to amply demonstrate the level of proficiency acquired in the use of the skills learned in the degree of Computer Engineering.

Keywords: Arduino, active redundancy by comparison, control system, microcontroller, redundant architecture

Índice

1	Introducción	11
1.1	Contexto	11
1.2	Objetivos	12
1.2.1	Implementación de un sistema de control	12
1.2.2	Diseño de una arquitectura redundante	12
1.2.3	Desarrollo del protocolo de comunicación serie	13
1.2.4	Implementación de software con funcionalidades específicas	13
1.2.5	Pasos a Seguir	13
1.3	Estructura de la memoria	14
2	Arquitectura del sistema	17
2.1	Diseño funcional del sistema	17
2.2	Diseño del tanque de reacción	19
2.2.1	Proceso químico simulado en el sistema	19
2.2.2	Sensores y actuadores	20
2.2.3	Protocolo de comunicación	21
2.2.4	Representación del tanque	22
2.2.5	Modificaciones a la vasija de reacción simulada	23
2.2.6	Librería SRTank . h	23
2.3	Diseño de la arquitectura redundante	24
3	Diseño de los buses de comunicación	27
3.1	Descripción del bus Serial Peripheral Interfase (SPI)	27
3.2	Asignación de roles SPI en el sistema redundante	29
3.3	Diseño de Custom SPI	30
3.4	Ajuste de la implementación real usando el osciloscopio	32
4	Diseño del mecanismo redundante de transporte	37
4.1	Protocolo de transporte	37
4.1.1	Funcionamiento	38
4.1.2	Temporizaciones	40

4.1.3	Ventajas y desventajas sobre SPI	40
4.1.4	Comunicación semi-simultánea con dos maestros	41
4.1.5	Uso del protocolo	42
4.2	Control de la redundancia del sistema	42
4.2.1	Máquinas de estados	43
4.2.2	Sincronización entre maestros	46
4.2.3	Recuperación tras un reinicio	47
4.2.4	Pérdida temporal de uno de los gemelos y resolución de errores	48
5	Implementación hardware del sistema	51
5.1	Descripción del sistema	51
5.2	Componentes hardware	52
5.2.1	Arduino Nano	52
5.2.2	Componentes electrónicos	53
5.2.3	Pantallas LCD	54
5.2.4	Anillo LED	54
5.3	Montaje del sistema	54
5.3.1	Diagrama y esquemático	58
5.4	Interfaz Gráfica de usuario	60
5.4.1	Implementación	62
5.5	Pruebas del sistema	63
5.5.1	Proceso químico completo	63
5.5.2	Forzar error por software	70
5.5.3	Reinicio manual de un controlador	70
5.5.4	Desconexión manual de un controlador	70
6	Estudio de la viabilidad de la simulación software del sistema	73
6.1	Ventajas del uso de un simulador	73
6.2	Modificaciones	74
6.2.1	Sockets TCP/IP	74
6.2.2	Shared memory	75
6.3	El nuevo periférico	76

6.3.1	Diseño	76
6.3.2	Pruebas	76
6.3.3	Conclusiones	78
6.4	Limitaciones	78
7	Conclusiones y Líneas Futuras	79
7.1	Conclusiones	79
7.1.1	Eficacia del sistema de control redundante	79
7.1.2	Uso del simulador	79
7.1.3	Implementación de la interfaz gráfica	79
7.2	Líneas futuras	80
7.2.1	Mejora del periférico en PICSimLab	80
8	Referencias	81
8.1	Listado de referencias	81
Apéndice A	Librería SRTank.h y Datasheets	85
A.1	Librería SRTank	85
A.2	LCD OLED 1.3" DataSheet	88
A.3	8 LEDs ring DataSheet	92
A.3.1	LED WS2812B	92
A.3.2	LEDs ring	95

Índice de figuras

Figura 1: Diseño funcional del sistema	18
Figura 2: Representación del tanque [7]	22
Figura 3: Serial Remote Tank	22
Figura 4: Serial Remote Tank modificado	23
Figura 5: Diseño de la arquitectura redundante	24
Figura 6: SPI con un único esclavo	28
Figura 7: SPI multipunto con tres esclavos	28
Figura 8: Comunicación SPI	29
Figura 9: Custom SPI con dos maestros	31
Figura 10: Señal de reloj	32
Figura 11: Señal de reloj y primera versión señal MOSI	33
Figura 12: Señal de reloj y señal MISO	34
Figura 13: Protocolo Custom SPI	39
Figura 14: Máquina de estados del controlador	44
Figura 15: Máquina de estados del comparador	45
Figura 16: Manejo de las comunicaciones por parte del comparador	45
Figura 17: Aviso de reinicio	48
Figura 18: Algoritmo de reinicio	48
Figura 19: Conexión entre dos Arduino Nano	55
Figura 20: Conexión entre tres Arduino Nano	55
Figura 21: Se añade un botón y LEDs	55
Figura 22: Reorganización de los componentes	55
Figura 23: Traslado del sistema a protoboards de mayor tamaño	56
Figura 24: Prueba de la pantalla LCD con un Arduino	56
Figura 25: Se añade al sistema la pantalla LCD	56
Figura 26: Se añaden al sistema más pantallas	56
Figura 27: Nueva pantalla para el comparador y LED de discrepancia	57
Figura 28: Resultado final del sistema	57
Figura 29: Diagrama final del sistema	58

Figura 30: Esquemático del sistema	59
Figura 31: Interfaz gráfica con el sistema apagado	60
Figura 32: Acción de reset	62
Figura 33: Acción de ERROR	62
Figura 34: Step 0, Llenar el tanque hasta 1500 litros	64
Figura 35: Step 0, Válvula de llenado abierta	64
Figura 36: Step X, Comprobar el llenado hasta los 1500 litros	64
Figura 37: Step X, Tanque en reposo mientras se realiza la comprobación	64
Figura 38: Step 1, Calentar el contenido del tanque	65
Figura 39: Step 1, Calefactor encendido	65
Figura 40: Step 2, Llenar el tanque completamente	66
Figura 41: Step 2, Válvula de llenado abierta	66
Figura 42: Step 3, Mezclar el contenido del tanque	66
Figura 43: Step 3, Agitador encendido	66
Figura 44: Step 4, Enfriar el contenido del tanque	67
Figura 45: Step 4, Enfriador encendido	67
Figura 46: Step 5, Vaciar el contenido del tanque	68
Figura 47: Step 5, Válvula de vaciado abierta	68
Figura 48: Step 6, Esperar la pulsación del botón para comenzar un nuevo proceso .	68
Figura 49: Step 6, Tanque en reposo	68
Figura 50: Nuevo periférico: cables entre instancias	77
Figura 51: Propiedades del periférico	77

Introducción

1.1. Contexto

En entornos industriales, es común el uso de microcontroladores para implementar el control de sistemas de producción. Estos sistemas se basan en programas que, a intervalos regulares, leen las mediciones de los sensores del sistema, toman decisiones de control basadas en estos datos y envían las órdenes correspondientes a los actuadores del sistema. Esta metodología permite un control automatizado y eficiente de diversos procesos industriales. Sin embargo, en sistemas críticos (como, por ejemplo, las vasijas de reacción en la industria química) no es aceptable que un fallo en el hardware o software del controlador provoque la parada del proceso, lo deje evolucionando libremente, o bloquee el sistema en un estado potencialmente peligroso. En este tipo de aplicaciones es esencial garantizar la tolerancia a fallos del sistema de control, es decir, la capacidad del sistema para continuar funcionando correctamente en caso de que ocurra un fallo en uno de sus componentes.

La introducción de redundancia es una solución común para lograr esta tolerancia a fallos, donde por redundancia se entiende la replicación, total o parcial, del sistema de control y el uso de un sistema comparador que coordine la operación de estas réplicas, y garantice que si uno de los controladores falla, otro pueda tomar su lugar sin interrumpir el proceso.

El problema que se aborda en este Trabajo de Fin de Grado (TFG) es la implementación de una arquitectura redundante por replicación y comparación para un sistema de control basado en Arduino Nano, para el gobierno de un proceso químico, representado por la simulación de un tanque de reacción. La arquitectura propuesta utiliza tres unidades de Arduino Nano, de las cuales dos actúan como controladores redundantes, mientras que la tercera funcionará como un comparador. Los dos Arduinos Nano que actúan como controladores gemelos reciben información de los sensores del tanque de reacción a través del comparador y ejecutan el

mismo algoritmo de control, tomando decisiones en función de los datos proporcionados por estos sensores, y enviando como respuesta órdenes a los actuadores del tanque de reacción. La duplicación del controlador asegura que, en caso de fallo de una de las copias, la otra pueda continuar controlando el proceso sin interrupciones. Por su parte el tercer Arduino Nano actúa como intermediario entre el tanque de reacción y los dos controladores redundantes, y su función principal es comparar estas respuestas para identificar discrepancias entre las salidas de los controladores. Si detecta una discrepancia significativa, el comparador determina cuál de los controladores está operando incorrectamente y descarta sus salidas, permitiendo que el controlador operativo mantenga el gobierno del proceso de reacción. Adicionalmente, si el comparador detecta un fallo aísla al controlador defectuoso, lo reinicia, mientras continua operando con el controlador válido, y volverá a incorporar al controlador reiniciado al esquema redundante una vez se haya recuperado.

El diseño del esquema redundante realizado para este TFG es escalable y puede ser adaptado a otros sistemas de control industrial que requieren alta disponibilidad y tolerancia a fallos, y puede ser implementado tanto sobre controladores Arduino Nano reales como simulados.

1.2. Objetivos

De acuerdo a la descripción del sistema redundante de control a implementar, los objetivos que debe conseguir este TFG son los siguientes:

1.2.1. Implementación de un sistema de control

Se desarrollará un sistema de control utilizando una placa Arduino Nano y comunicaciones serie que sea capaz de monitorizar y controlar de manera estable un proceso físico/químico en un tanque de reacción simulado. Este sistema debe ser capaz de leer las mediciones de los sensores, procesar los datos y enviar las órdenes correspondientes a los actuadores del tanque.

1.2.2. Diseño de una arquitectura redundante

Se creará una arquitectura redundante que incluya una copia secundaria del sistema de control basado en Arduino Nano, operando en paralelo con la copia primaria y utilizando el mismo algoritmo de control. Además, se diseñará un módulo comparador, también basado en

Arduino Nano, que recibirá los comandos de control de ambos controladores mediante buses serie, los comparará para verificar su corrección y retransmitirá los comandos válidos a los dispositivos de control del tanque.

1.2.3. Desarrollo del protocolo de comunicación serie

Se desarrollará un protocolo de comunicación serie basado en *Serial Peripheral Interface* (SPI), con la característica de poder usar cualquier pin digital del Arduino Nano para la comunicación y, que además permita a un sistema esclavo trabajar con dos maestros de manera casi simultánea.

1.2.4. Implementación de software con funcionalidades específicas

Utilizando entornos de desarrollo Arduino y Python, se implementará un software que proporcione las siguientes funcionalidades:

- Control del proceso en el tanque de reacción por uno de los Arduinos.
- Sincronización y comparación de las respuestas de control redundantes procedentes de los dos Arduinos, y ejecución de acciones correctoras en caso de detectar discrepancias.
- Desarrollo de una interfaz de usuario que permita la monitorización y seguimiento de la acción de control, así como la introducción de errores en el sistema para comprobar la eficacia de los mecanismos de redundancia.

1.2.5. Pasos a Seguir

Los pasos seguidos para el completo desarrollo del proyecto han sido los aquí descritos:

- **Planteamiento del sistema en un entorno simulado utilizando el software PIC-SimLab:** PICSimLab [1] es un laboratorio de simulación para placas de desarrollo basadas en microcontroladores Microchip de las familias PIC y ATmega, incluyendo periféricos simulados. Este paso se ha usado para comenzar a estructurar el diseño final del sistema, y sus componentes esenciales. Aunque finalmente se ha optado por una implementación real sobre hardware (por limitaciones de PICSimLab), el uso de simulador en

estas fases iniciales permitió la familiarización con el entorno de desarrollo Arduino sin complicaciones de cableado ni conectividad entre ordenador y placa de desarrollo.

- **Creación del protocolo de comunicación serie basado en SPI:** Este paso ha sido necesario para diseñar la conexión del comparador (que actúa como esclavo) con los dos controladores (que actúan como maestros). Teniendo en cuenta que no se puede usar la implementación SPI hardware del microcontrolador porque, al necesitar conectar dos maestros al comparador, se necesita también usar el doble de pines de su microcontrolador, el ATmega328p, y por tanto, no basta con los pines predefinidos para SPI.
- **Implementación real utilizando protoboards, cableado y Arduinos reales:** Una vez que el trabajo en simular ha llegado a su fin, el siguiente paso es hacer una implementación física real del sistema, que evita ciertas limitaciones del sistema simulado, y que permite demostrar la correcta operación del diseño especificado para el sistema.
- **Desarrollo de la interfaz gráfica de usuario:** Para permitir un manejo más ameno y sencillo del sistema durante sus pruebas, se ha desarrollado una aplicación software que muestre cual es el estado del sistema, y cual ha sido su evolución, todo ello de forma muy visual y eficiente.

1.3. Estructura de la memoria

Tras el presente Capítulo 1, que da un contexto, una motivación y una serie de objetivos a este TFG, se describirán en el Capítulo 2 los diseños funcionales, del sistema, del tanque de reacción y de la arquitectura redundante. En el Capítulo 3 se verá el diseño de los buses de comunicación, incluyendo su monitorización con un osciloscopio durante las pruebas. En el Capítulo 4 se detallan las partes del diseño redundante, tanto para el protocolo de comunicación como el control de la redundancia. La implementación física, incluyendo los componentes hardware, el montaje, la interfaz gráfica y las pruebas del sistema quedan reflejados en el Capítulo 5. En el Capítulo 6 se detalla el trabajo realizado con el simulador PICSIMLab, incluyendo el diseño de un nuevo periférico para comunicaciones, y sus modificaciones y limitaciones. En el Capítulo 7 se presentan las conclusiones del proyecto y las posibles líneas futuras de investigación y desarrollo. Finalmente, se incluye el listado de referencias utilizadas a lo largo

del documento, mientras en el apéndice se incluyen el listado de la librería SRTank . h y los *DataSheets* de la pantalla LCD y del anillo de LEDs.

2

Arquitectura del sistema

En este capítulo se describen las características funcionales y arquitectura diseñada tanto para el sistema a controlar (el tanque de reacción) como para el sistema controlador. En particular:

- El tanque de reacción, siendo el núcleo del proceso químico a controlar, requiere un diseño detallado que incluya no solo los aspectos físicos y operativos, sino también el conjunto de sensores y actuadores que serían necesarias para obtener un control preciso del proceso. En particular, la interacción del controlador con estos sensores y actuadores será fundamental para monitorear y ajustar en tiempo real los parámetros de control del tanque.
- Para garantizar que el sistema pueda continuar operando de manera segura y eficiente, incluso ante fallos imprevistos, es imprescindible que la arquitectura diseñada presente una arquitectura redundante, que no solo mejore la fiabilidad del sistema, sino que también proporcionen mecanismos de recuperación que minimizan el impacto de cualquier fallo y aseguren la continuidad del proceso.

2.1. Diseño funcional del sistema

Como se ilustra en la Figura 1 el sistema está compuesto por cuatro módulos funcionales: dos copias idénticas del sistema controlador, un comparador, y un tanque de reacción simulado.

En particular, en la Figura 1 se representa el diagrama de bloques del diseño funcional del sistema, incluyendo las interacciones entre los controladores gemelos y el comparador, y la conexión de éste último con el tanque de reacción simulado. El diagrama también mues-

tra qué datos son intercambiados entre estos módulos, y las interacciones entre los diversos componentes.

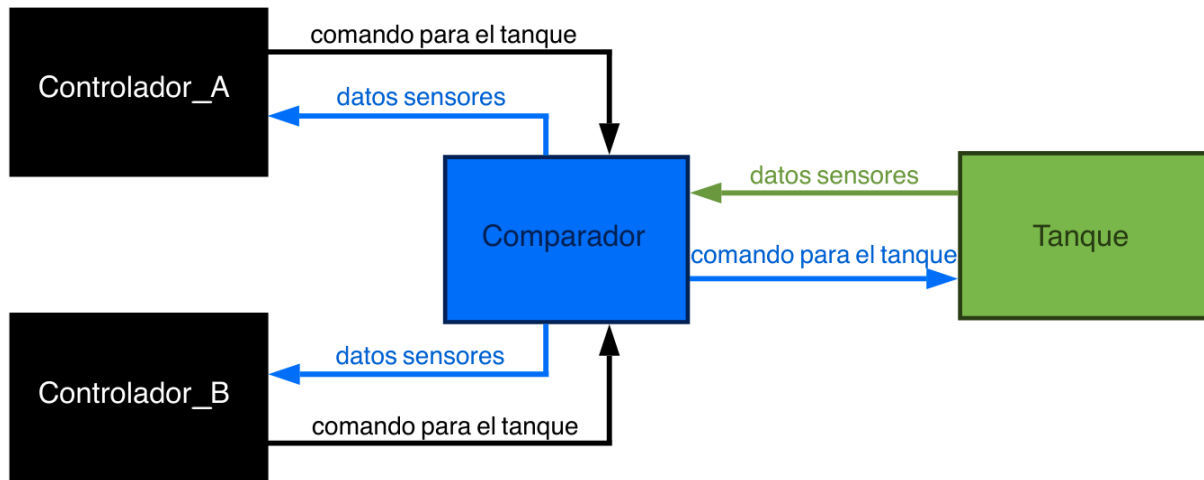


Figura 1: Diseño funcional del sistema

En particular, las especificaciones para las interacciones entre estos componentes son las siguientes:

1. **Controladores gemelos:** Las dos instancias del controlador reciben datos de los sensores del tanque y envían comandos a los actuadores del mismo. Las entradas recibidas por estos controladores incluyen las lecturas de los sensores de temperatura y nivel del tanque, junto a una serie de flags que permiten comprobar el correcto funcionamiento del tanque. Las salidas generadas por los controladores son los comandos para los actuadores del tanque (calentador, enfriador, agitador y válvulas de entrada y salida).
2. **Comparador:** El comparador recibe las salidas (es decir, los comandos) generados por ambos controladores gemelos y las compara para detectar discrepancias. En operación normal, la actuación del comparador es invisible para el resto de integrantes del sistema, pero si se detecta una discrepancia, el comparador decide cuál de los controladores está funcionando incorrectamente, y al compararlas toma medidas correctivas, (como el reinicio del controlador defectuoso). Las entradas al comparador son los comandos de los controladores gemelos y los datos procedentes de los sensores del tanque, y sus salidas son los comandos enviados al tanque, y las copias de los datos de los sensores que van a ser enviadas a los controladores.

3. **Tanque de reacción simulado:** Este módulo representa el proceso químico que se necesita controlar, y que es una reacción de saponificación (es decir, fabricación de jabón). Este proceso se desarrolla dentro de un tanque de reacción (simulado), que incluye sensores para medir la temperatura, el nivel de líquido, las revoluciones del motor del agitador y una electrónica básica que genera un conjunto de flags que indican si el nivel ha llegado al mínimo o al máximo volumen, si se ha llegado a la máxima o mínima temperatura, y si el motor del agitador está o no encendido. La electrónica del tanque contiene también actuadores para controlar el calentador, el enfriador, el agitador y las válvulas del tanque. El tanque recibe como entradas los comandos de los controladores, y las salidas del tanque son los datos de los sensores que se envían de vuelta a los controladores a través del comparador.

2.2. Diseño del tanque de reacción

El tanque de reacción es uno de los componentes principales del sistema, en el que se lleva a cabo el proceso químico. Dado que, obviamente, no podemos trabajar con un tanque real, se ha usado una simulación software de tanque y proceso químico. Para esta simulación se ha partido del simulador de tanque de reacción que viene con la distribución del programa PICSimLab [1], pero para cumplir con todas las necesidades del sistema se ha modificado el código fuente de dicho simulador para añadir un conjunto de nuevos sensores. Además, para facilitar el control de dicho tanque, también se ha desarrollado una librería llamada SRTank . h.

2.2.1. Proceso químico simulado en el sistema

El proceso químico que se realiza en la simulación es conocido como ‘saponificación’. La saponificación es un proceso químico que consiste en la reacción de una base (usualmente sosa cáustica, NaOH) con un ácido graso (aceites o grasas) para producir jabón y glicerina. Este proceso ha sido utilizado durante siglos para la fabricación de jabón y sigue siendo un método fundamental en la industria de productos de limpieza y cuidado personal.

Durante la saponificación, los triglicéridos presentes en los aceites reaccionan con el hidróxido de sodio, rompiendo las moléculas de grasa y formando sales de ácidos grasos (jabón) y glicerol. La reacción es exotérmica, lo que significa que libera calor, aumentando la tempe-

ratura de la mezcla.

El proceso de saponificación comienza con la apertura de la válvula de llenado del tanque, donde se introduce una solución de agua y sosa cáustica (NaOH) hasta alcanzar los 1500 litros. (Cada paso del proceso incluye una comprobación intermedia antes de continuar con el siguiente paso). Una vez la solución de agua y sosa cáustica está en el tanque, se enciende el calentador para elevar la temperatura de la solución a 35°C. Con la solución a esta temperatura se llena el tanque con aceite hasta alcanzar los 3000 litros, proporcionando los ácidos grasos necesarios para la reacción.

Una vez llenado el tanque, el agitador se enciende para mezclar la solución de sosa cáustica con los aceites, iniciando así la reacción de saponificación, que aumenta la temperatura de la mezcla debido a su naturaleza exotérmica. Para controlar este incremento de temperatura, se conecta un enfriador, que devolverá la mezcla a la temperatura inicial de 25°C. Tras un tiempo de espera para asegurar la completa saponificación, se abre la válvula de vaciado del tanque para transferir el jabón resultante a la maquinaria de procesamiento, donde será dividido y empaquetado en sus presentaciones finales. Después de completar todos estos pasos, el sistema entra en un estado de espera hasta que el usuario pulse el botón que reinicia el proceso.

2.2.2. Sensores y actuadores

La aplicación Serial Remote Tank (SR Tank), es un programa diseñado para simular un tanque que puede ser controlado remotamente utilizando comunicación serial; la versión original de este programa ha sido creada por el mismo desarrollador de la herramienta PICSimLab [1], y está incluida en la distribución de PICSimLab.

El tanque tiene varios sensores y actuadores que pueden ser leídos y controlados usando comandos en texto ASCII enviados mediante el protocolo de comunicación serie. Los parámetros del puerto de comunicación serial deben ser 19200 8N1.

Este programa puede ser controlado desde un microcontrolador real o virtual y; adicionalmente, durante la simulación hay una realimentación visual en la que puede apreciarse como el tanque se llena, vacía, caliente, enfría o se agita en tiempo real gracias a una animación.

Como salidas digitales, el SR Tank tiene un flag para el flotador (niveles alto y bajo), y para la temperatura máxima y mínima. Por otro lado, como salidas analógicas, el tanque proporciona datos de volumen y temperatura.

Como entradas digitales, el SR Tank tiene una válvula de entrada y salida, un calentador, un enfriador y un agitador. Por otro lado, como entradas analógicas, hay un nivel de activación de alarma de temperatura máxima y un nivel de activación de alarma de temperatura mínima.

2.2.3. Protocolo de comunicación

La comunicación con el tanque se realiza usando los siguientes comandos:

- Escribir en Entrada Digital: Se envía un byte en formato hexadecimal 0x0N donde N es el número de entrada, seguido de un segundo byte con el valor 0x00 para desactivar o 0x01 para activar. Como ejemplo, la activación de la entrada 2 requiere la secuencia de dos escrituras:

```
Serial_write(0x02);  
Serial_write(0x01);
```

- Leer Salida Digital: Se envía un byte en formato hexadecimal 0x1N donde N es el número de salida y se lee un byte. El byte leído tiene el valor 0x00 para indicar dispositivo inactivo o 0x01 para dispositivo activo. Como ejemplo, la lectura de la salida 3 requiere la siguiente secuencia:

```
Serial_write(0x13);  
valor = Serial_read(0);
```

- Escribir en Entrada Analógica: Se envía un byte en formato hexadecimal 0x2N donde N es el número de entrada, seguido de dos bytes para escribir un valor de 16 bits. Como ejemplo, para escribir el valor 230 en la entrada analógica 1, se usaría esta secuencia:

```
Serial_write(0x21);  
valor = 230;  
Serial_write((valor & 0xFF00) >> 8);  
Serial_write(valor & 0x00FF);
```

- Leer Salida Analógica: Se envía un byte en formato hexadecimal 0x3N donde N es el número de la salida, y se leen entonces dos bytes para formar el valor de 16 bits. Como ejemplo, para leer la salida analógica 2 se requiere esta secuencia:

```
Serial_write(0x32);
valorh = Serial_read(0);
valorl = Serial_read(0);
valor = (valorh << 8) | valorl;
```

2.2.4. Representación del tanque

La Figura 2 muestra una representación esquemática de un tanque de reacción similar al representado en la reacción. La Figura 3 muestra el interfaz gráfico de usuario de la aplicación simulador, con los parámetros de simulación estándares y los checks para determinar qué dispositivos existen en el tanque, así como la representación gráfica del estado del tanque.

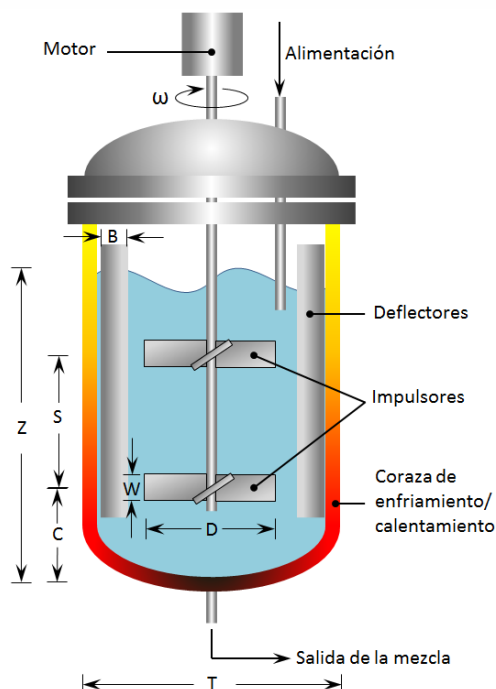


Figura 2: Representación del tanque [7]

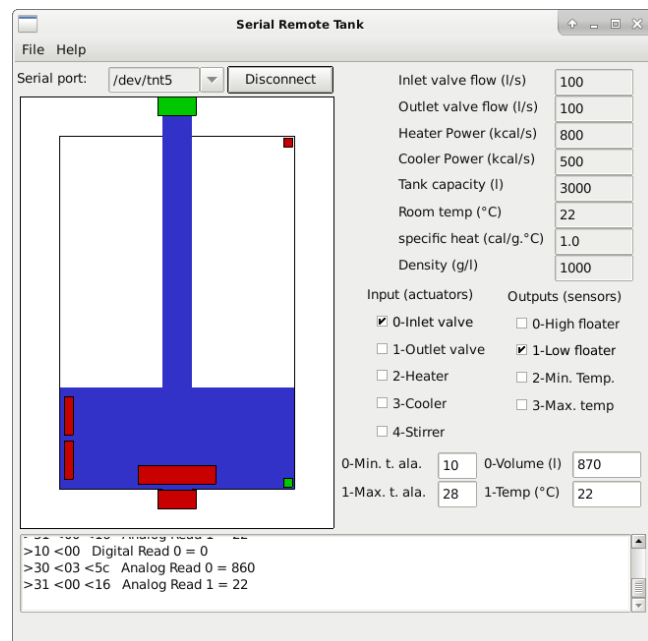


Figura 3: Serial Remote Tank

2.2.5. Modificaciones a la vasija de reacción simulada

Como parte del TFG se ha modificado el código fuente de la aplicación Serial Remote Tank para añadir las siguientes funcionalidades:

- Se ha añadido al listado de ‘Outputs (sensors)’, un nuevo flag de verdadero o falso que informa si el motor del agitador está o no encendido, y que aparece como el cuarto integrante ‘4-Revolutions’.
- También se ha agregado a esa misma sección un cuadro de texto, llamado ‘2-Rev(rpm)’, para mostrar las revoluciones (revoluciones por minuto) en tiempo real del motor.

El resultado de estos cambios se muestra en la Figura 4, donde ya se cuenta también con los parámetros finales de simulación usados.

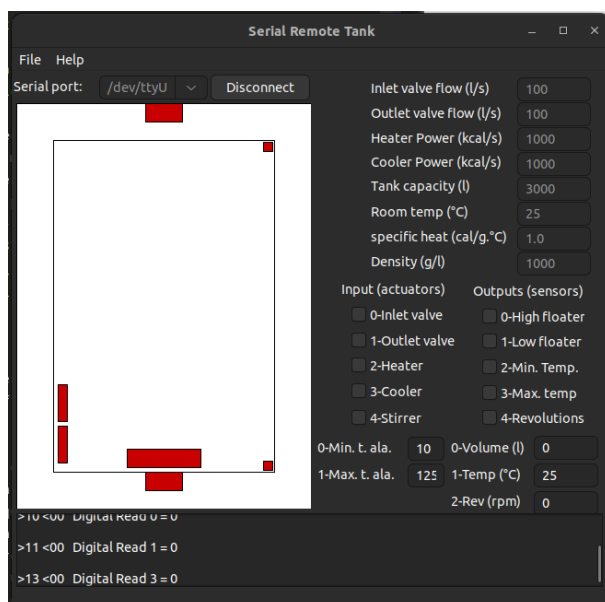


Figura 4: Serial Remote Tank modificado

2.2.6. Librería **SRTank.h**

Para facilitar el trabajo con el simulador SR Tank, se ha desarrollado una librería específica que permite el control completo del mismo. Esta librería, **SRTank.h**, proporciona una serie de funciones que simplifican la interacción con los diferentes componentes del tanque, como son las válvulas, el calentador, enfriador y el agitador. Todas las funciones disponibles en esta librería están listadas en el apartado A.1 del Apéndice A, en el que se ofrece una descripción

detallada de cada función, permitiendo a los usuarios entender y utilizar todas las capacidades del tanque de manera efectiva.

2.3. Diseño de la arquitectura redundante

La arquitectura redundante del sistema está diseñada para garantizar la continuidad y fiabilidad del control del proceso de reacción, incluso en presencia de fallos en el módulo de control.

Como muestra la Figura 5, esta arquitectura consta de dos instancias de un módulo controlador, y de un comparador. Los módulos de control están constituidos por dos microcontroladores idénticos (uno en cada módulo) que ejecutan el mismo algoritmo de control y que están conectados al tanque de reacción a través del comparador. Su función es tomar decisiones de control basadas en los datos de los sensores y enviar comandos de control al tanque.

El comparador, que también está implementado usando un microcontrolador, recibe los comandos de salida de ambos controladores y compara para detectar cualquier discrepancia. Si se detecta una discrepancia, el comparador identifica el controlador defectuoso y envía un comando de reinicio a dicho controlador. Además, el comparador puede mostrar señales de control de error para alertar sobre cualquier fallo detectado, asegurando que el sistema mantenga su operatividad; si con este reinicio no consigue resolver el problema, el comparador detendrá la evolución del proceso químico, llevándolo a un estado seguro.

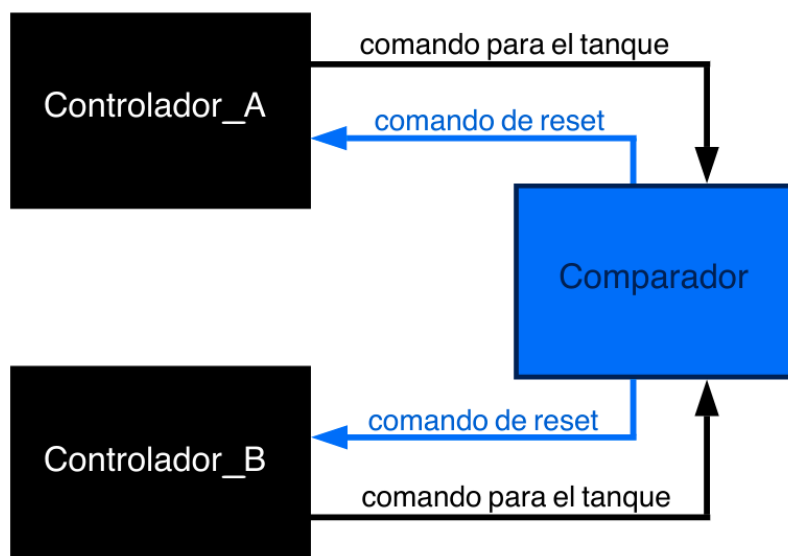


Figura 5: Diseño de la arquitectura redundante

Sobre este diseño de arquitectura redundante es necesario comentar que la cantidad de redundancia en el sistema se ha limitado para ajustar el trabajo (y hardware) necesario a los requerimientos de un TFG; en particular, se puede observar que el comparador, al no estar redundado, es un potencial punto único de fallo. Obviamente, en un sistema real de producción la arquitectura del sistema incluiría redundancia también para el comparador.

Diseño de los buses de comunicación

Una comunicación eficiente y robusta entre los diferentes componentes del sistema es imprescindible para su correcto funcionamiento. En esta sección, se muestra el diseño de los buses de comunicación utilizados en el proyecto y cómo dan solución a esa necesidad.

El capítulo comienza describiendo el bus estándar SPI (Serial Peripheral Interface), tras lo cual se describen las mejoras y adaptaciones realizadas a dicho bus para satisfacer las necesidades de nuestro sistema. Finalmente, se describen las pruebas realizadas para comprobar el correcto funcionamiento de este nuevo bus (al que llamaremos Custom SPI), utilizando un osciloscopio para ajustar y verificar las señales de comunicación sobre el hardware real.

3.1. Descripción del bus Serial Peripheral Interface (SPI)

El Serial Peripheral Interface (SPI) es un estándar de interconexión utilizado para transferir información entre dispositivos electrónicos, a través de un bus serie síncrono usando una arquitectura maestro-esclavo que permite comunicaciones bidireccionales.

El bus SPI está compuesto por cuatro líneas: una línea de reloj (SCLK), una línea de datos entrantes (MISO en el caso del maestro y MOSI en el caso del esclavo), una línea de datos salientes, (MOSI en el caso del maestro y MISO en el caso del esclavo), y un pin de selección de chip (SS), que permite al maestro identificar y activar el esclavo con el que quiere intercambiar datos. La Figura 6 muestra el esquema típico de conexiones para un sistema bus SPI con un único maestro y esclavo; en la Figura 7 se muestra cómo se implementa un sistema con tres esclavos, usando líneas de datos multiplexadas y tres líneas individuales de selección de esclavos.

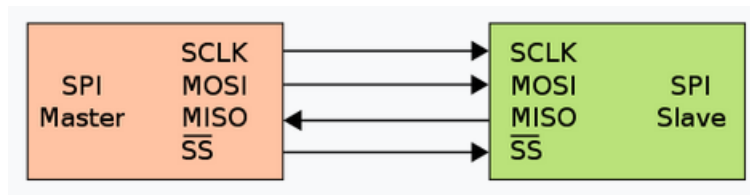


Figura 6: SPI con un único esclavo

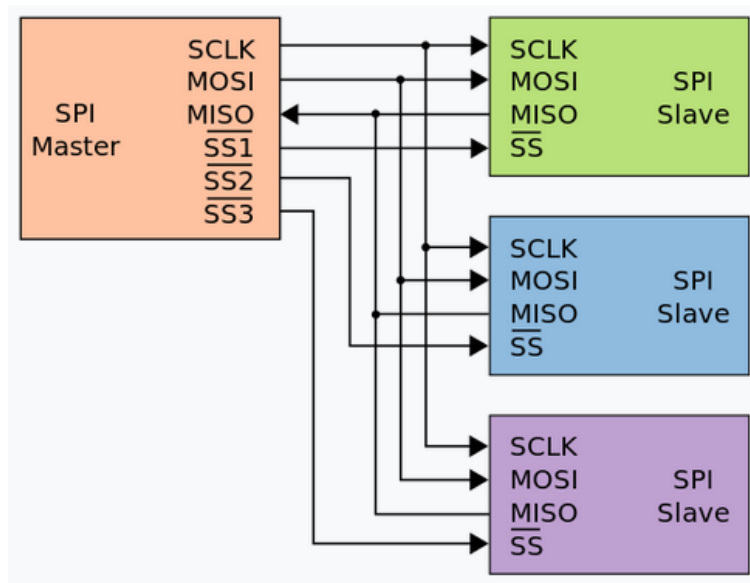


Figura 7: SPI multipunto con tres esclavos

El protocolo de señalización sigue la siguiente secuencia:

1. El maestro SPI genera una señal de reloj (SCLK) que sincroniza la transferencia de datos y selecciona al esclavo activando la línea SS correspondiente.
2. Los datos se transfieren en ambos sentidos simultáneamente a través de MOSI (Master Out Slave In) y MISO (Master In Slave Out), sincronizados con los flancos del reloj. Usualmente los datos se envían en los flancos de bajada del reloj.
3. El esclavo recibe los datos enviados por el maestro (8 bits) y, a su vez, puede enviar datos de vuelta, ya que en el mismo momento que el maestro escribe en MOSI, está leyendo lo que el esclavo ha escrito en MISO, así se consigue utilizar el envío de un ACK como dato válido.

4. La comunicación continúa hasta que el maestro desactiva la línea SS, indicando al esclavo que la transferencia ha terminado.

En la Figura 8 se muestra en ejemplo de comunicación SPI.

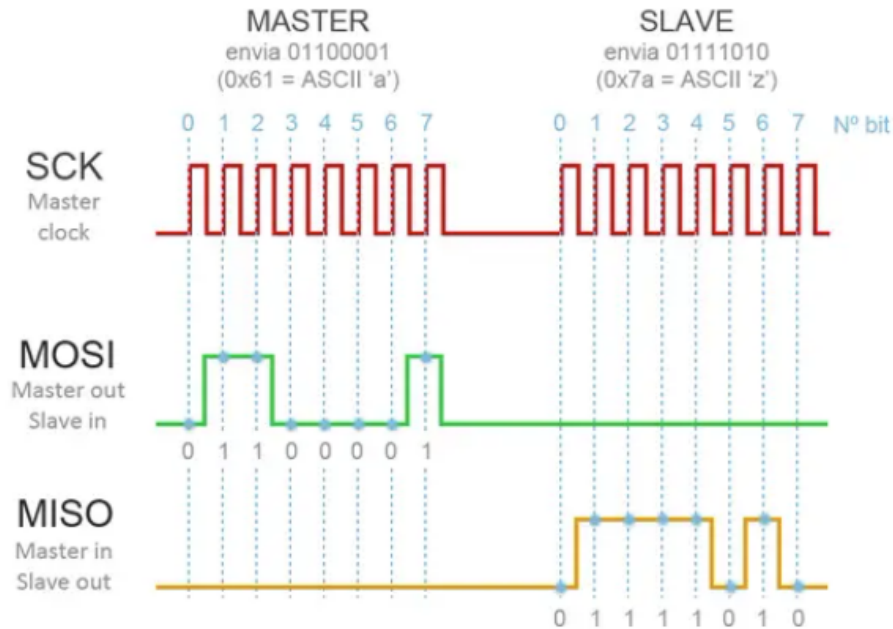


Figura 8: Comunicación SPI

3.2. Asignación de roles SPI en el sistema redundante

En nuestro sistema redundante se ha elegido usar el protocolo SPI porque requiere solo tres líneas para transferir datos, y es muy sencillo de implementar por software, por lo que es adecuado para usar las líneas de GPIO de los microcontroladores en los controladores y el comparador.

Dado que en el sistema que se quiere desarrollar debe comunicarse cada controlador con el comparador, en un principio se pensó en configurar al comparador como maestro y a los controladores gemelos como esclavos. Sin embargo, esta configuración no era adecuada: los esclavos no pueden iniciar comunicación, sino solo responder a requerimiento del maestro, por lo que, con ese esquema, el comparador debía estar constantemente iniciando comunicaciones con los controladores para proporcionarles información sobre el estado del tanque y obtener el comando que el controlador quiera ejecutar.

Un problema de este esquema radica en que la temporización del sistema no la controla el

sistema que toma decisiones y envía las órdenes, por lo que potencialmente se crean desajustes (por ejemplo si el controlador necesita enviar un comando con urgencia, pero debe esperar a que el comparador se comunique con él, o por ejemplo si un comando está listo para ser enviado, pero su transmisión se realiza cuando el comparador lo permite, lo que se puede traducir en que el comando llegue al tanque demasiado tarde). Sin embargo, el mayor problema con diferencia de esta disposición es la sincronización de los controladores redundantes, ya que sería el propio comparador el que debiera controlarla en lugar de gestionarse entre los propios controladores.

Debido a los razonamientos anteriores, se ha optado por un esquema de dos controladores configurados como máster SPI y un comparador como slave. Para poder implementar esta configuración ha sido necesario editar el protocolo para crear una nueva versión customizada que permita la existencia de dos instancias diferenciadas SPI en el slave; estos cambios son bastante sustanciales, por lo que se presentarán en la siguiente sección.

3.3. Diseño de Custom SPI

Custom SPI es una implementación totalmente software del protocolo SPI, diseñada para superar las limitaciones del soporte hardware del microcontrolador. Al no depender del hardware específico de SPI, Custom SPI proporciona una mayor libertad en la asignación de pines, permitiendo utilizar cualquier pin digital disponible en el microcontrolador para las señales SPI. Esta flexibilidad es crucial para permitir la operación concurrente del comparador con los dos controladores, usando dos instancias distintas de SPI. Además, es especialmente útil en configuraciones donde los pines dedicados de SPI están ocupados o no sean accesibles, y es crucial para permitir la operación concurrente de dos instancias de SPI.

En un sistema estándar SPI, los pines dedicados para MOSI, MISO, SCLK y SS están predefinidos por el hardware del microcontrolador. En el caso del Arduino Nano, estos son D11, D12, D13 y D10. Sin embargo, Custom SPI permite que cualquier pin digital sea asignado a estas funciones, proporcionando una mayor flexibilidad en el diseño del sistema. En este proyecto, los controladores usan los pines 7, 8, 9 y 10, mientras que el comparador usa los pines 10, 11, 12 y 3 para el controlador-A, y los pines 7, 8, 9 y 2 para el controlador-B, respectivamente.

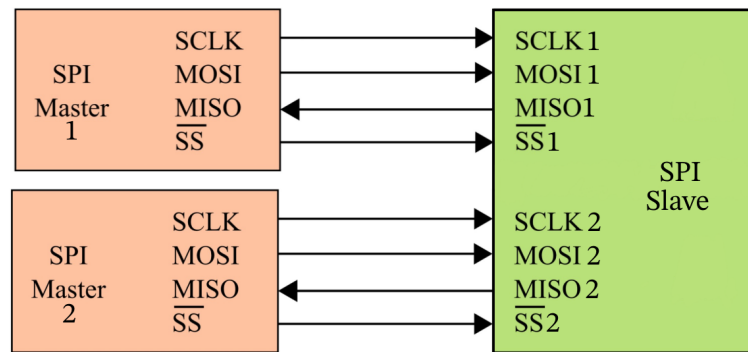


Figura 9: Custom SPI con dos maestros

Los cambios a nivel de bus realizados en Custom SPI en comparación con el SPI estándar se muestran en la Figura 9, mientras que a nivel de torre de protocolos, estos cambios permiten la operación concurrente de las dos instancias de SPI y se detallan a continuación:

- Capa Física - Asignación flexible de pines: A diferencia de SPI estándar, Custom SPI permite usar cualquier pin digital para MOSI, MISO, SCLK y SS. Esto facilita la configuración del sistema cuando los pines específicos de SPI no están disponibles, se necesitan para otros fines o son requeridos más de cuatro pines.
- Capa de Enlace de Datos:
 - Interrupciones por software: Custom SPI utiliza interrupciones por software para gestionar la comunicación. Esto permite al comparador responder las solicitudes de comunicación de ambos maestros casi simultáneamente, de manera eficiencia y con una reducida latencia.
 - Manejo de múltiples maestros: Cada maestro tiene su propio conjunto de pines para la comunicación con el comparador. Esto evita colisiones en el bus y facilita la concurrencia, permitiendo que ambos maestros se comuniquen con el comparador sin interferencias.
- Capa de Transporte - Control de temporización: La generación del reloj y la sincronización de datos se realizan mediante software, lo que permite ajustar la frecuencia del reloj y la temporización según las necesidades del sistema. Esto es necesario para asegurar que ambas instancias de Custom SPI operen de manera sincrónica y eficiente.

- Capa de Aplicación - Eliminación de restricciones: El nuevo enfoque de Custom SPI hace posible el desarrollo de sistemas en los que un esclavo se comunique simultáneamente con dos maestros, eliminando una de las restricciones principales de SPI estándar. Esto permite diseñar arquitecturas más complejas y flexibles, mejorando la capacidad de adaptación del sistema a diversas necesidades, (por ejemplo, una posible evolución del sistema actual con triple redundancia en la que, el comparador como esclavo, se comunique con tres másteres Custom SPI).

3.4. Ajuste de la implementación real usando el osciloscopio

Para asegurar el correcto funcionamiento del protocolo Custom SPI y ajustarlo para que sea robusto y eficiente, se ha utilizado un osciloscopio Tektronix, modelo TDS 220 para monitorizar la señalización durante la comunicaciones, detectar problemas de temporización, y ajustar el comportamiento. A continuación, se detallan los pasos y observaciones realizadas durante este proceso:

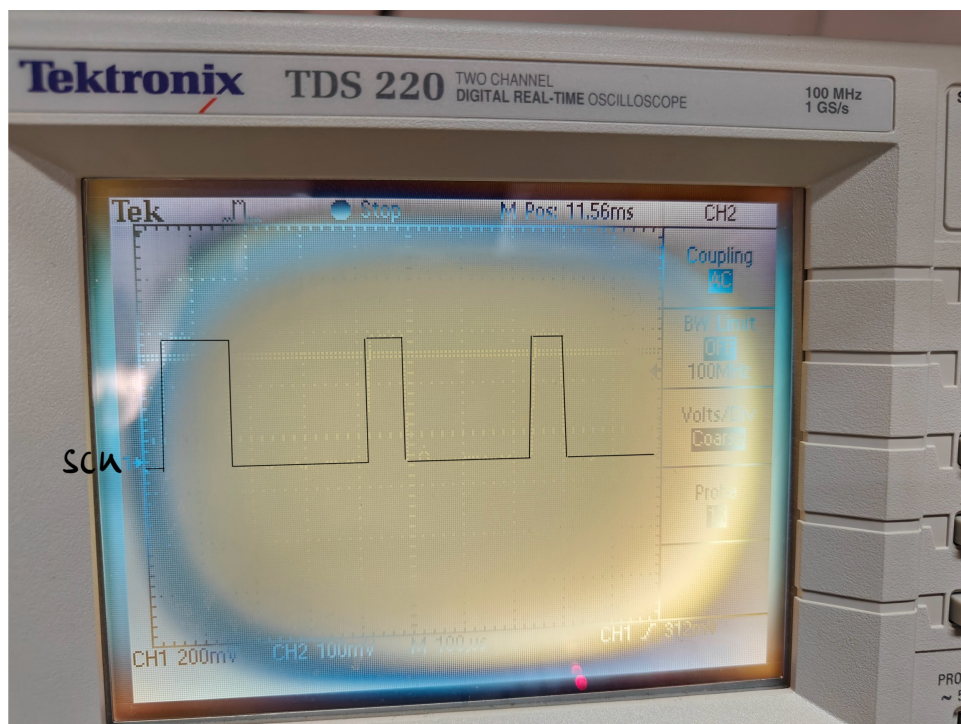


Figura 10: Señal de reloj

1. Verificación inicial de la señal de reloj: El primer paso ha consistido en comprobar que la señal de reloj (SCLK) generada por el software oscilaba correctamente. Para ello, se conectó el osciloscopio a la línea SCLK y se observó su comportamiento en tiempo real. Como se ilustra en la Figura 10, se pudo comprobar que la señal de reloj oscilaba correctamente y que se generaban los ocho pulsos necesarios para transmitir un byte como 8 bits individuales.

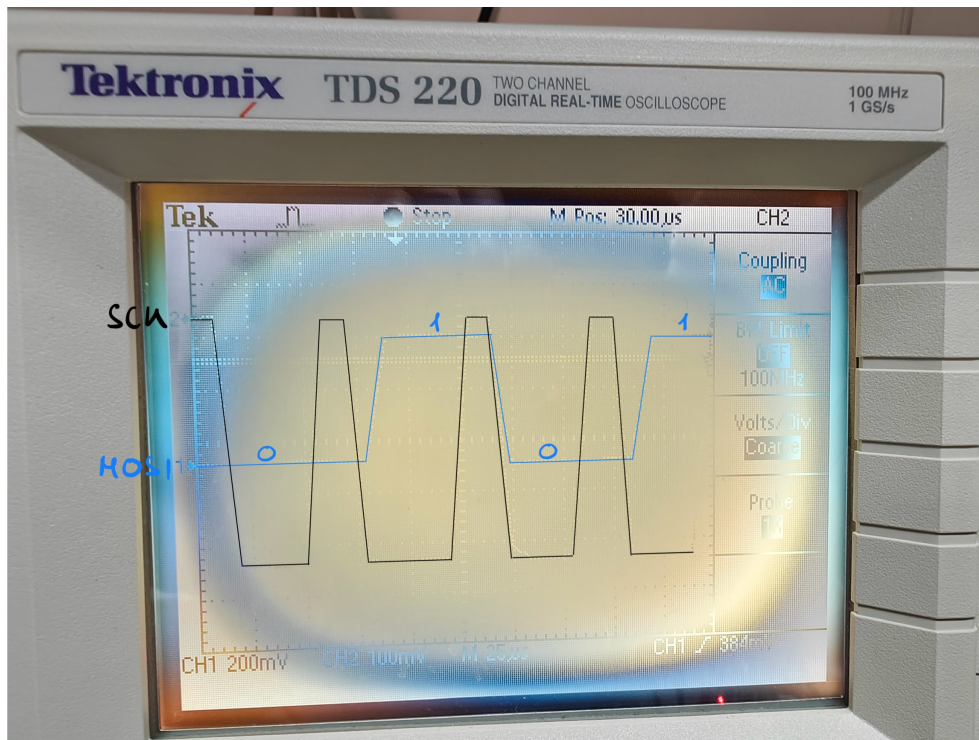


Figura 11: Señal de reloj y primera versión señal MOSI

2. Comprobación de la sincronización de señales de reloj y datos: El paso siguiente fue la visualización simultánea de la señal de reloj (SCLK) y la señal de datos (MOSI) en el osciloscopio para analizar su sincronización. Como muestra la Figura 11, se comprobó que en la implementación inicial había problemas, porque en ciertos momentos la temporización no era lo suficientemente precisa para garantizar la correcta lectura de los datos. Se identificaron una serie de desajustes en la sincronización entre los flancos del reloj y los cambios en la señal de datos, resultando en que había momentos en los que los datos no estaban listos para ser leídos en el flanco de bajada del reloj, lo que se traducía en

lecturas incorrectas. Esto se aprecia en la Figura 11 cuando la señal MOSI cambia del valor '0' al valor '1'.

3. Ajuste de la temporización: Para solucionar estos problemas se modificó la implementación del protocolo. Primero se aseguró que los cambios lógicos en la señal de datos ocurren junto a los flancos de bajada del reloj, momento en el que se comienza la lectura del dato. También se incrementó el tiempo que permanece a 0 la señal de reloj; esto se hizo para garantizar que el receptor tiene suficiente tiempo para leer los datos de manera estable, ya que en la implementación original ocurría que un mismo valor podía ser leído dos veces o ninguna, (es decir la lectura de una secuencia 1-0-1-0, a veces, daba 1-0-0-0 ó, 1-1-1-0). Adicionalmente, el tiempo que permanece a 1 la señal de reloj, también requirió un pequeño ajuste para que tenga la mínima duración necesaria, ya que sirve únicamente como delimitador de bits.

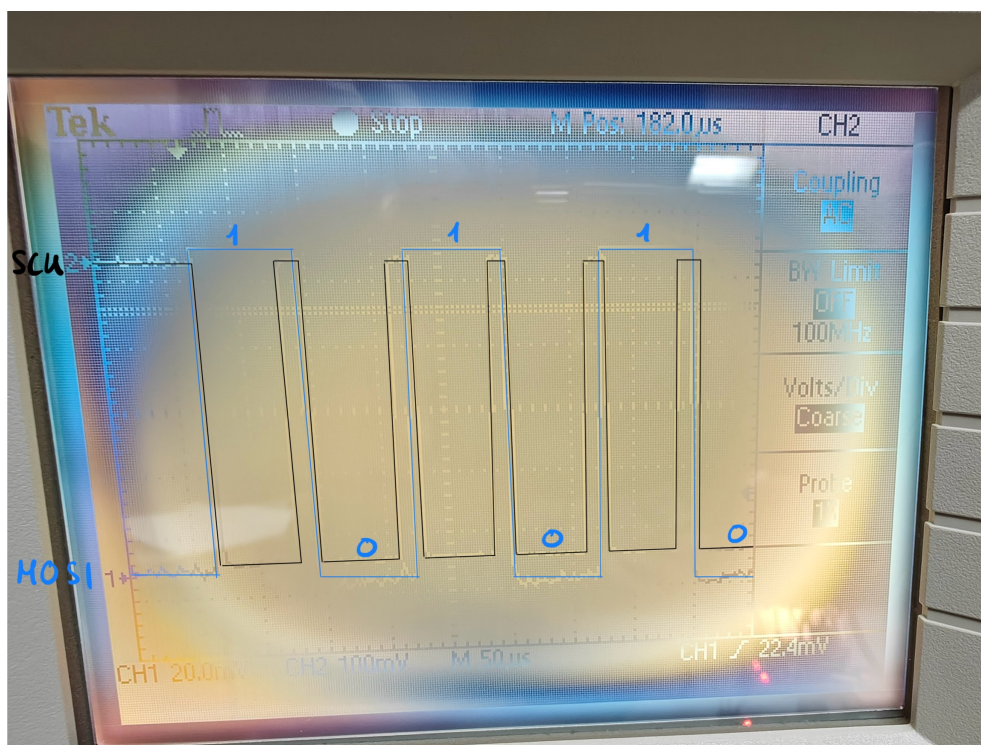


Figura 12: Señal de reloj y señal MISO

Por último, se modificó el inicio de la lectura para que se realice una espera antes de comenzar; en la Figura 12 se puede ver como la señal de reloj está a 1 al inicio un tiempo

mayor que en el resto de los pulsos. Esta modificación fue crucial para solventar el mayor fallo detectado, que consistía en que el dato enviado del slave al máster siempre era leído como la mitad del dato real, (es decir, por ejemplo, se enviaba el número 170, en binario 1010 1010, y se recibía el 84, en binario 0101 0100), ya que el receptor no leía el primer bit y se rellenaba con '0' el bit más significativo, (SPI transmite los bits comenzando por el bit más significativo). Gracias a esta solución, se proporciona tiempo suficiente al slave para comenzar la transmisión de datos manteniendo la integridad de los mismos, pero sin repercutir en el rendimiento del protocolo.

4

Diseño del mecanismo redundante de transporte

En este capítulo se describe el diseño e implementación de dos aspectos clave: el protocolo de transporte y el control de la redundancia usando dicho protocolo.

En primer lugar, se detalla el protocolo de transporte, que diseñado, ha sido desarrollado para proporcionar una mayor flexibilidad y funcionalidad que el SPI estándar. Este protocolo permite una comunicación más adaptable entre los componentes del sistema, utilizando cualquier pin digital disponible y facilitando la interacción con múltiples maestros.

En segundo lugar, se presenta la implementación de los mecanismos de control de la redundancia, usando este protocolo de transporte. En particular, los mecanismos de sincronización y recuperación aseguran que el sistema puede mantenerse operativo y resolver cualquier inconsistencia de manera eficiente.

4.1. Protocolo de transporte

Después de haber desarrollado en el capítulo 3 el diseño y las características del bus de comunicación Custom SPI, ahora se profundizará en el diseño del protocolo de transporte que se utiliza sobre dicho bus. Este protocolo, aunque similar al estándar SPI, ofrece mejoras en términos de flexibilidad y funcionalidad, ya que permite a los controladores comunicarse con el comparador utilizando las mismas señales que el SPI estándar, pero con la flexibilidad añadida de poder usar cualquier pin digital, e incluye mecanismos de interrupción y temporización precisos para asegurar una comunicación robusta y eficiente; estos mecanismos garantizan

que los datos se transmiten y reciben correctamente, incluso cuando hay múltiples maestros involucrados.

4.1.1. Funcionamiento

El comportamiento del nuevo protocolo es similar a SPI, ya que sigue siendo compatible con él y mantiene las cuatro líneas entre los componentes de la comunicación. No obstante, debido a que no usa el módulo hardware SPI del microcontrolador, el desempeño es distinto, especialmente en lo relativo a la arquitectura del esclavo.

En este proyecto los controladores, (que solo se comunican con un esclavo, el comparador), utilizan las mismas cuatro señales que SPI, pero mapeadas sobre cualquiera de los pines digitales de GPIO (`MasterSPI.h`). Sin embargo el comparador implementa dos esclavos, por lo que necesita poder comunicarse con cada controlador por cuatro líneas distintas, haciendo un total de ocho pines necesarios, e independientes de los pines usados por el hardware SPI del micro. Para permitir esta configuración se ha desarrollado una librería (`SlaveSPI.h`) que emula la operación de dicho hardware especializado, pero permite mapear las señales sobre cualquier pin digital de GPIO.

Para la comunicación de un controlador con el comparador, primero se definen y conectan cuatro pines en cada micro, correspondientes a las cuatro señales necesarias para el protocolo. Después, y como punto especialmente relevante, es necesario configurar en el comparador una interrupción hardware asociada al pin de SS, para que se dispare cuando el pin tenga un valor de cero, ya que por defecto esa línea siempre tiene el valor 1 (está en alto) y únicamente se pone a cero (en bajo) cuando el máster selecciona al slave para realizar una comunicación.

Una vez se ha activado la interrupción y, por tanto es conocido el deseo del maestro de realizar una comunicación, el controlador (en el rol de maestro) realiza una pequeña espera para dar tiempo al comparador a prepararse para la comunicación, tras lo cual el máster genera una señal de reloj SCLK de ocho pulsos en cada uno de los cuales se enviará y recibirá un bit, dando como resultado un byte, que es la unidad de transmisión de datos. En cada flanco de bajada de la señal SCLK el maestro activará o desactivará su línea MOSI para indicar si envía un '1' o un '0', y de igual manera el slave hará lo mismo con su línea MISO.

Por temas de eficiencia, únicamente se modifica la señal de MOSI o MISO si hay un cambio con respecto al anterior dato, es decir, si la línea se encontraba en alto indicando el envío de

un '1', y se quiere volver a enviar un '1', la línea no será modificada, mientras que si se quisiera enviar ahora un '0', cuando se detecte un pulso de bajada del reloj se pondría en bajo la señal.

Un punto importante a destacar es que, debido a la existencia de dos líneas separadas de datos (MOSI y MISO), el protocolo es bidireccional y full-duplex.

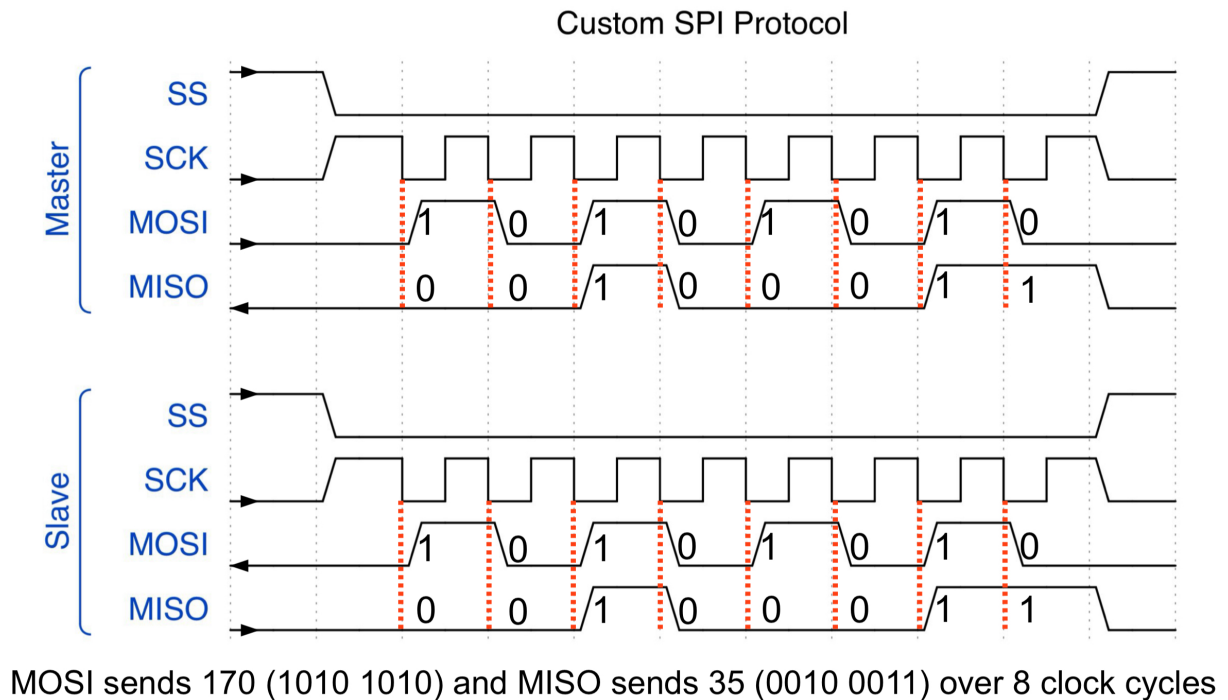


Figura 13: Protocolo Custom SPI

En la Figura 13 se observa un ejemplo del funcionamiento del protocolo. El máster envía el dato 170, (que en binario corresponden con 1010 1010), y en cada pulso de reloj envía uno de esos bits de datos a través de la línea MOSI. Cuando la señal de reloj SCLK oscila y genera un flanco de bajada, se envía el bit correspondiente, (en este ejemplo, al no haber dos bits iguales seguidos, la señal va oscilando los valores '1' y '0'). Por su parte, el slave lee dicha señal para guardar el valor recibido, mientras que por su línea MISO está realizando la misma operación que el máster; en este ejemplo el esclavo quiere enviar el dato 35, (correspondiente con 0010 0011 en binario); a diferencia de la señal MOSI que oscilaba debido a que se enviaba un '1' y después un '0', MISO en este ejemplo solamente necesita modificar su estado en el momento de enviar el tercer y séptimo bit.

4.1.2. Temporizaciones

A diferencia del SPI hardware, donde el reloj (SCLK) es generado automáticamente por el hardware del microcontrolador, en el protocolo Custom SPI el reloj es generado por software, lo que implica que la frecuencia del reloj depende de la implementación del software y de la velocidad a la que el microcontrolador puede alternar el estado del pin de reloj. Esto permite ajustar la frecuencia del reloj según las necesidades del sistema y las capacidades del microcontrolador. El maestro SPI es responsable de generar las transiciones de SCLK, asegurando que cada ciclo del reloj tenga la duración adecuada para que los datos sean leídos correctamente tanto por el maestro como por el esclavo.

La línea SS debe activarse bajando el nivel lógico antes de iniciar la transmisión de datos y desactivarse después de completar la comunicación. La temporización de esta línea es indispensable para asegurar que el esclavo está listo para recibir o enviar datos, y es necesario que el master haga una pequeña espera tras activarla para dar tiempo al slave a prepararse para la comunicación.

4.1.3. Ventajas y desventajas sobre SPI

El nuevo protocolo descrito proporciona una nueva manera de trabajar que, en comparación al estándar SPI, mejora en los siguientes puntos:

- Flexibilidad en la selección de pines: A diferencia del SPI hardware, que usa forzosamente cuatro pines del micro especificados por el fabricante para las líneas (SCLK, MOSI, MISO, SS), Custom SPI permite la configuración del bus SPI sobre cualquier pin digital del microcontrolador.
- Múltiples maestros en el mismo sistema: Se facilita la implementación de sistemas con múltiples maestros, ya que cada maestro puede tener su propio conjunto de pines para la comunicación con el esclavo. Esto reduce el riesgo de colisiones en el bus y permite una mayor flexibilidad en la arquitectura del sistema. Se consigue también un arbitraje más simple, ya que la gestión del acceso al esclavo puede implementarse en software, permitiendo un control más granular y personalizado sobre la coordinación de múltiples maestros.

- Control de la temporización: La frecuencia del reloj (SCLK) puede ajustarse en software, lo que permite adaptarse a diferentes velocidades según las necesidades de los dispositivos conectados y las limitaciones del sistema. También se permite el ajuste de la temporización de la transmisión y recepción de datos con mucha precisión, lo que se traduce en la posibilidad de optimizar el rendimiento del bus manteniendo la integridad de los datos.
- Escalabilidad y portabilidad: Se puede implementar en cualquier microcontrolador que tenga pines digitales disponibles, lo que hace que el protocolo sea muy escalable y portátil entre diferentes plataformas de hardware.

Obviamente, la implementación por software implica también ciertas desventajas: al dejar de usar el hardware específico para SPI y cambiarlo por software se introduce una mayor latencia en la comunicación, (debido al procesamiento necesario para la ejecución de código para generar las señales de reloj y gestionar la transmisión de datos), y también se aumenta un poco la carga en el procesador. Sin embargo la implementación realizada ha conseguido temporizarlo todo de manera suficientemente eficaz para que el slave sea capaz de comunicarse simultáneamente con ambos maestros, requiriendo únicamente un desfase de ocho milisegundos entre la comunicación con el primer máster que inicie el protocolo y el segundo máster.

4.1.4. Comunicación semi-simultánea con dos maestros

Como se acaba de comentar, y tratándose de un bus serie, la comunicación cuando los dos maestros quieren transmitir al mismo slave debe ser primero con uno, y después con el otro, por eso uno de los maestros se activa después que su gemelo, para evitar que ambos se comuniquen exactamente en el mismo momento. Realmente la cifra de ‘delay’ podría disminuirse al máximo sin que la integridad de los datos enviados corriera peligro, ya que la robustez y velocidad del Custom SPI podría soportar una mayor ajuste, pero el problema reside en las interrupciones externas: si las dos interrupciones llegan al mismo tiempo, (es decir, ambos maestros activan su propio SS, generando cada uno una interrupción, y las dos interrupciones son capturadas al mismo tiempo por el esclavo), podría suceder que la rutina de servicio de la interrupción no se llegara a ejecutar completamente para alguno de los maestros, resultando en la no comunicación con el mismo. Es por esta razón por lo que se ha implementado un

retardo único de 8 ms entre los controladores gemelos, que es lo suficientemente alto para que no se interfieran entre sí, y lo suficientemente pequeño para que el sistema no vea su operación comprometida.

4.1.5. Uso del protocolo

Para manejar de manera más sencilla la transmisión de datos entre Arduinos a través de Custom SPI se ha creado una librería C++ que utiliza el nuevo protocolo para realizar la comunicación. Para usarla simplemente se deben definir los pines correspondientes a las cuatro señales, inicializar dichos pines y después, crear un objeto de la clase MasterSPI o SlaveSPI. Ambos objetos tienen incorporada la función `transfer()` que, según si es un maestro o un esclavo, podrá ser llamada en cualquier momento, o solo si se ha activado la línea SS indicando el inicio de una comunicación.

Estas librerías cuentan también con la opción de usar el método `transfer16()`, que permite transmitir un dato que ocupe dos bytes, es decir, dieciséis bits. Por último, también incorporan la funcionalidad de `setClockDivider()` heredado de SPI, que modifica el retardo entre oscilaciones del reloj del protocolo para el ajuste del mismo por parte del programador de una manera sencilla.

4.2. Control de la redundancia del sistema

Asegurar la fiabilidad y la continuidad del sistema es indispensable, y el control de la redundancia juega un papel crucial en esto; esta sección explora cómo las máquinas de estados finitos de los controladores y del comparador gestionan eficazmente las comunicaciones y las operaciones del sistema. Se incluyen los algoritmos que garantizan que los controladores estén siempre sincronizados, así como los mecanismos que permiten al sistema recuperarse tras un reinicio o una pérdida de un controlador. Además, se detallan las estrategias que el sistema utiliza para resolver errores generales, garantizando que cualquier fallo sea manejado de manera eficiente y segura.

4.2.1. Máquinas de estados

El algoritmo del controlador ha sido desarrollado en lenguaje C++ y sigue los siguientes pasos:

1. Espera la pulsación de un botón para iniciar el proceso.
2. Una vez se da comienzo, el controlador debe sincronizarse con su gemelo.
3. Inicia comunicación con el esclavo y envía un comando al tanque.
4. Inicia una segunda comunicación en la cual solo recibe datos, en este caso, el estado del tanque (volumen, temperatura y los flags).
5. Comprueba la respuesta del slave, la cual debe ser un valor especificado como *acknowledgement* (ACK). Si no es así, quiere decir que se trata de un comando de *reset* que se verá en detalle en la sección 4.2.3.
6. Con los datos recibidos, se selecciona el siguiente comando a enviar y se cambia de estado de acuerdo a la máquina de estados mostrada en la Figura 14.
7. Se imprime por la terminal serie el estado del tanque y el comando que se va a enviar.
8. Se muestra en la pantalla OLED el step del proceso en el que está el controlador y se hace una pequeña espera antes de volver a empezar.
9. Cuando se completa el proceso, en la selección del comando existe un step bloqueante que detendrá la ejecución del algoritmo hasta que se registre de nuevo una pulsación en el botón.

El algoritmo del comparador se basa en un muestro periódico continuo del estado del tanque para tener los datos actualizados y listos para ser enviados en cuanto el controlador lo requiera. Cuando alguno de los controladores genere (activando SS) una interrupción para realizar una comunicación, el manejador de la interrupción levantará un flag para que en el bucle principal se lleve a cabo la comunicación, siempre que sea posible. La máquina de estados de la comunicación del comparador con los dos controladores queda reflejada en la Figura 16.

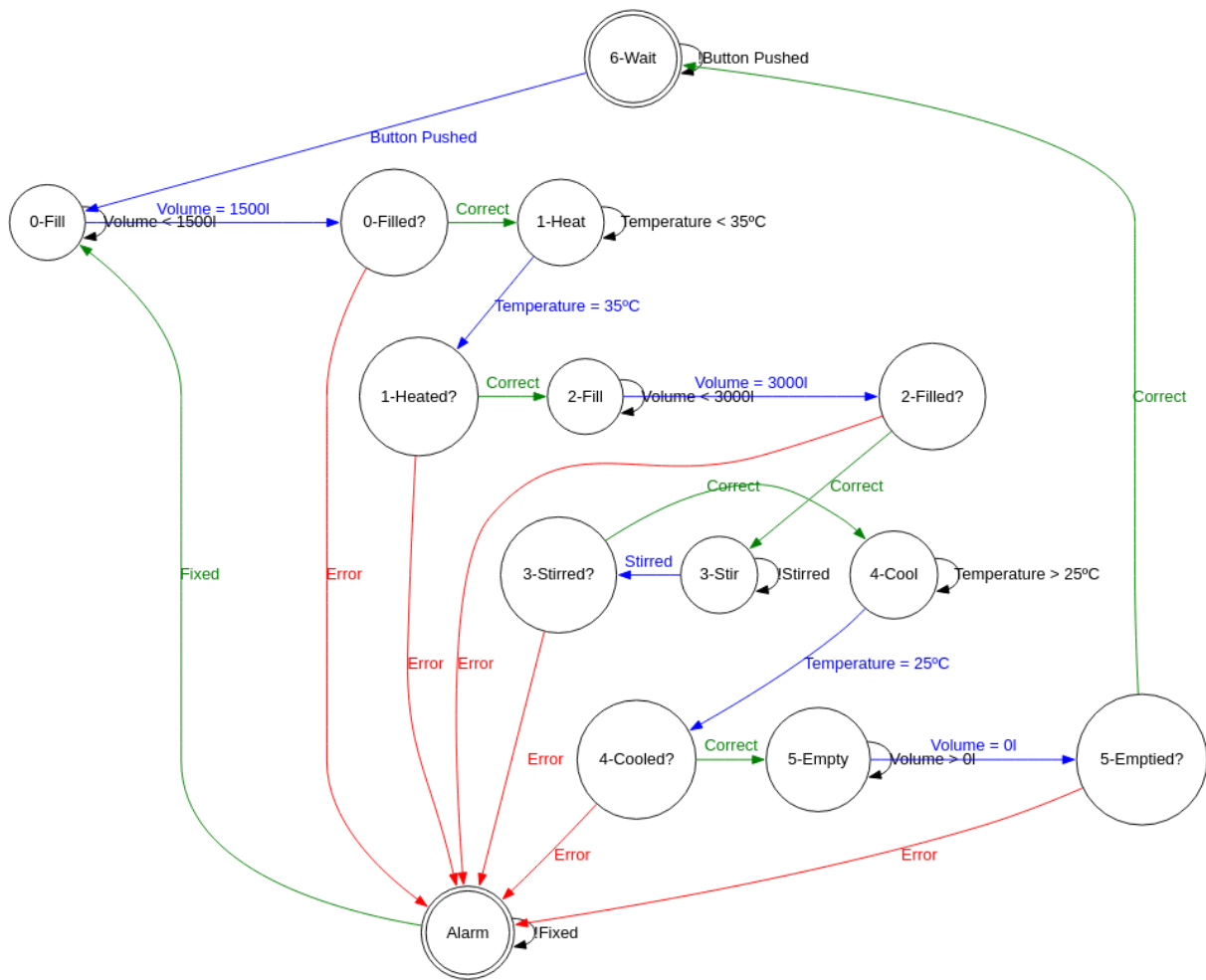


Figura 14: Máquina de estados del controlador

Una vez se han recibido los dos comandos, si estos son iguales (o si solo se recibe uno), se procede a ejecutarlo directamente y a cambiar de estado según la Figura 15. Cuando dichos comandos son distintos, el comparador enciende el LED y el anillo LED para indicar que se ha detectado una discrepancia. Dependiendo del estado en el que se encuentre el proceso de control del tanque (Fill, Stir, Cool, etc.), el comparador conoce cuáles son los posibles comandos que puede recibir, (por ejemplo si está en el estado de enfriar, los posibles comandos aceptables son encender o apagar el cooler, o el comando de error). El comparador puede discriminar qué comando ejecutar y a qué controlador debe mandar un comando de reset. Una vez se ha enviado la orden al tanque, se procede a resetear los flags utilizados y a actualizar, si es necesario, el estado mostrado en la pantalla. El comparador continuará su bucle principal muestreando el estado del tanque hasta que se vuelva a solicitar una interrupción por parte

de algún controlador.

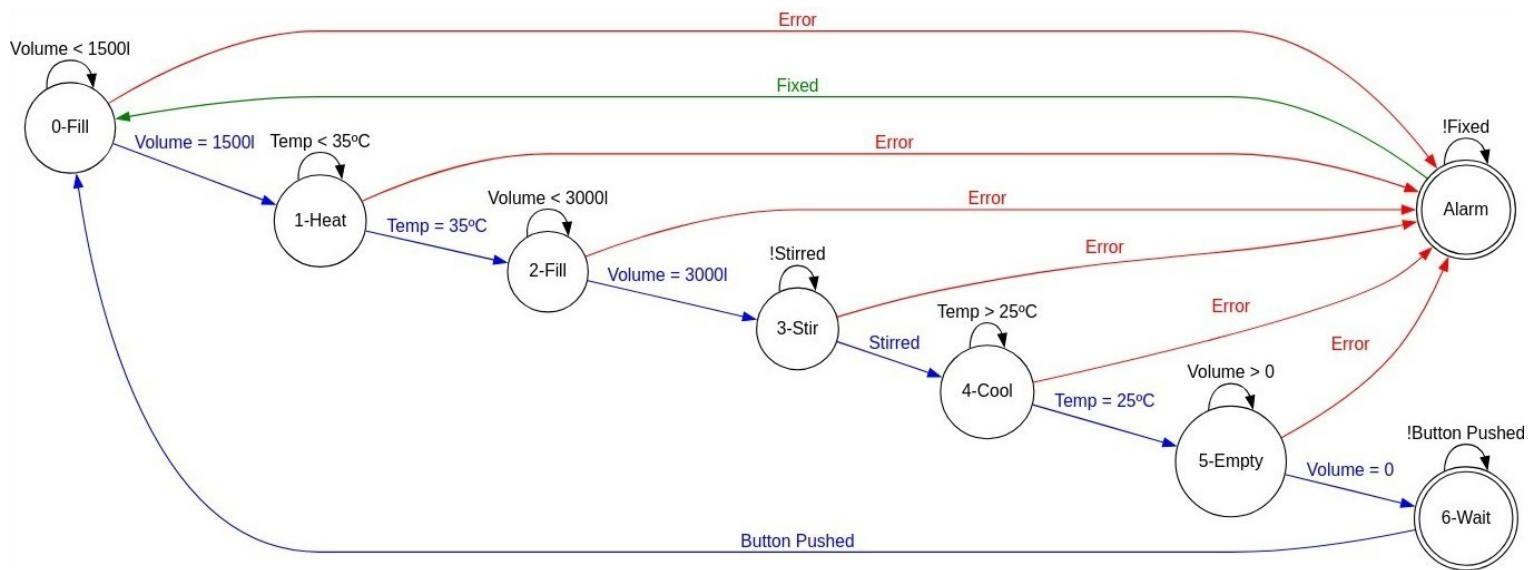


Figura 15: Máquina de estados del comparador

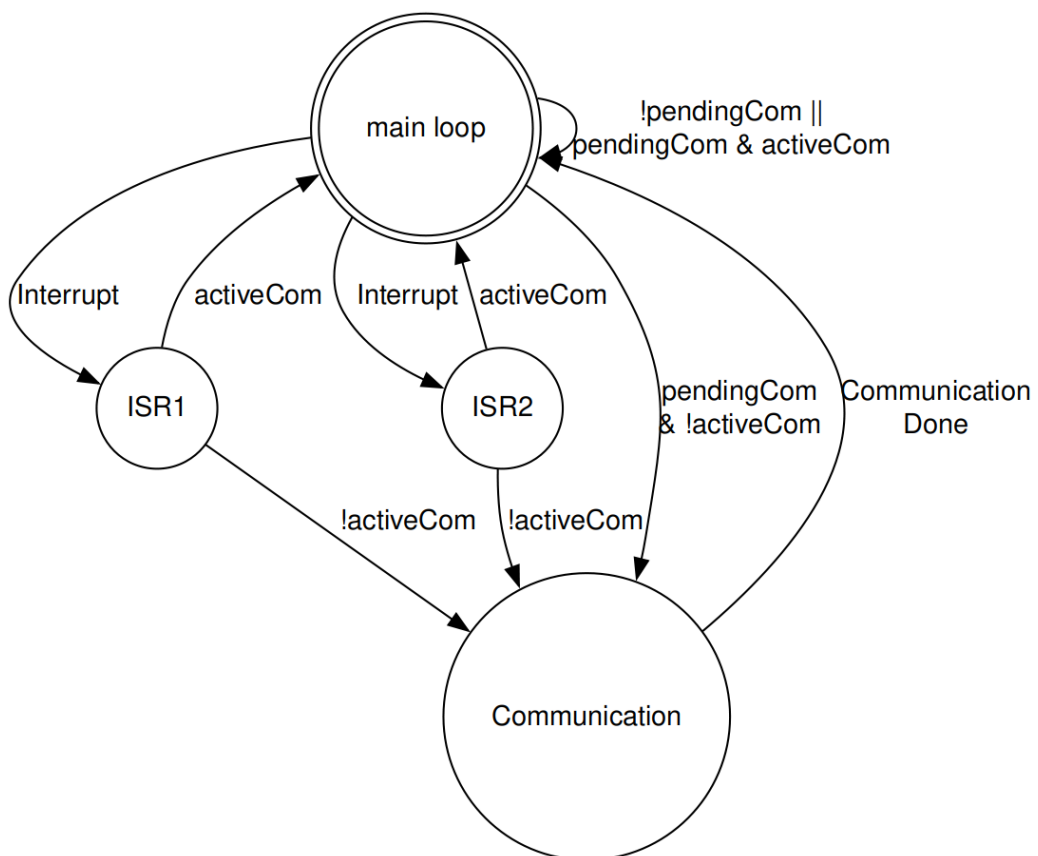


Figura 16: Manejo de las comunicaciones por parte del comparador

4.2.2. Sincronización entre maestros

El uso de dos controladores hace necesario mecanismos que eviten posibles desincronizaciones entre ambos, para lo cual se ha incluido una línea extra por cada controlador de forma que los controladores están conectados entre sí usando dos pines unidireccionales (líneas *READY*), a través de los cuales se gestiona la sincronización. Concretamente, el mecanismo de sincronización debe garantizar que ambos controladores envíen al mismo tiempo el mismo comando al comparador, de forma que el comparador pueda comparar su contenidos y detectar posibles discrepancias. El algoritmo de sincronización del controlador funciona de la siguiente manera:

1. Recepción y análisis de datos: El controlador recibe los datos del estado del tanque, los analiza y decide qué comando debe enviar al comparador.
2. Activación de la línea *READY*: Cuando el controlador está listo para iniciar la comunicación con el comparador, activa su línea *READY* y verifica el estado de la línea *READY* proveniente de su gemelo.
3. Verificación del estado de la línea: Si la línea está activa indica que el otro controlador había preparado su comando primero y está esperando. En este caso, se procede inmediatamente a la comunicación. En cambio, si la línea está baja significa que el otro controlador aún no tiene listo su comando, por lo que deberá esperar la llegada del otro controlador.

Este mecanismo presenta el desafío de que si uno de los controladores no llega a preparar su comando debido a un apagón, reinicio continuo, u otra circunstancia, el controlador que estaba esperando la llegada del gemelo, debe continuar con la ejecución para no bloquear el sistema. Por ello, el sistema usa un temporizador de time-out y si tras un segundo y medio el otro controlador no ha activado su línea *READY*, el controlador procederá a la comunicación para evitar el bloqueo del sistema debido a la pérdida de uno de los gemelos.

En un principio se pensó en realizar la sincronización de otra manera, con el comparador como orquestador. Sin embargo, se descubrió que si uno de los controladores iniciaba la comunicación, generando una interrupción externa en el comparador, y este último lo bloqueaba durante la espera de la llegada del otro controlador, se ocasionaba una desincronización del

protocolo de comunicación. Esto ocurría porque el maestro iniciaba la comunicación pero era el esclavo (comparador) el que bloqueaba. Si el gemelo tardaba más de lo normal en llegar, se desincronizaba el protocolo, causando problemas en la transmisión de datos. Por eso se dedujo que la manera más sencilla y segura de sincronizar ambas comunicaciones era hacerlo antes de llevarlas a cabo.

4.2.3. Recuperación tras un reinicio

En el momento en el que el comparador detecta una discrepancia entre los comandos recibidos, intentará identificar al infractor (usando la información de contexto del proceso controlado). Una vez ha decidido qué controlador es el que ha generado un comando erróneo, forzará su sincronización enviándole en la siguiente comunicación el step actual del sistema. Dicho valor, distinto del ACK (que indica que está todo correcto), informa al controlador de que debe reiniciarse, y de cuál es el step actual del sistema, para que, tras reiniciarse, el controlador comience directamente por dicho step. En ese momento, el controlador mostrará por pantalla un mensaje avisando del reinicio, y enciende su LED de reinicio para una mejor visualización del hecho de que se está produciendo esta resincronización; la Figura 17 muestra el aspecto del hardware del controlador durante este proceso.

El mayor reto de este reinicio ha sido conseguir que el controlador, una vez reiniciado, se sincronice con el sistema saltando directamente al step actual. El problema principal reside en que el controlador está configurado para encenderse y activarse cuando detecte una pulsación en un botón; por ello, para evitar el bloqueo del controlador reiniciado sin tener que volver a pulsar manualmente el botón de arranque, se ha añadido una nueva línea física desde el comparador hacia los controladores, denominada ‘línea de reset’. La activación de este pin avisa al controlador de que no debe esperar al botón, sino que debe saltar al step indicado por el comparador cuando le envió el comando de reset. Por tanto, el comparador debe poner en alto la línea de reset de un controlador cuando envíe el comando de reset y desactivarla una vez lograda la resincronización.

Por otro lado, en la Figura 18, se puede distinguir que el estado del controlador puede pasar del estado ‘Reset’ al estado de ‘Wait Button’, lo que ocurre cuando hay un error fatal en el que, aunque el comparador reinicia al controlador, el sistema finalmente no se puede recuperar y se detiene.

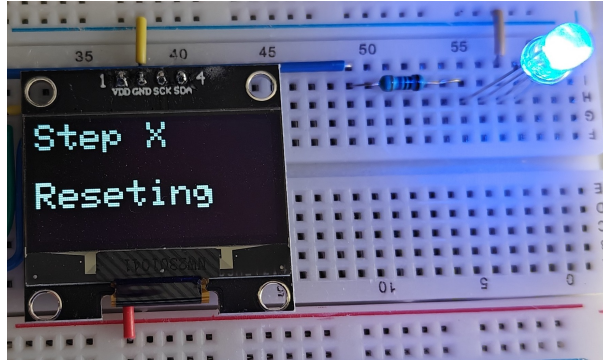


Figura 17: Aviso de reinicio

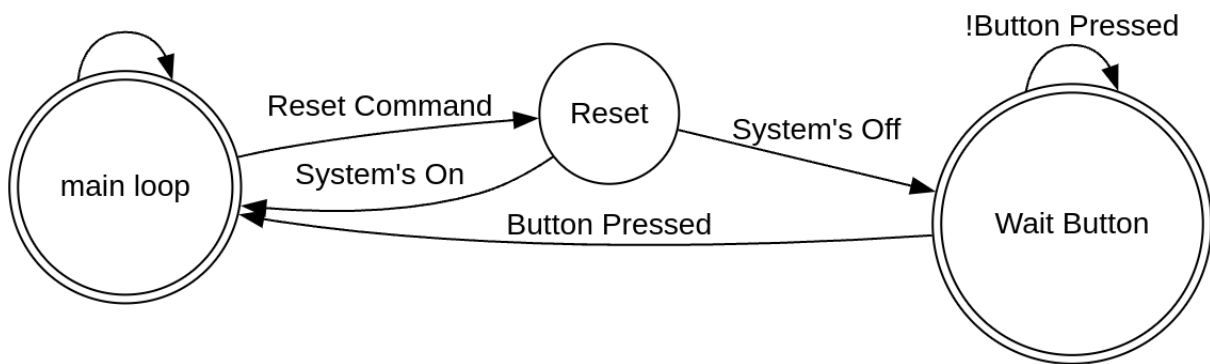


Figura 18: Algoritmo de reinicio

4.2.4. Pérdida temporal de uno de los gemelos y resolución de errores

En caso de una pérdida temporal de uno de los controladores gemelos, el sistema está diseñado para continuar operando usando solo el controlador restante mientras intenta recuperar el controlador perdido. El comparador supervisa continuamente ambos controladores y, si detecta que uno de ellos no responde, se activa un protocolo de espera. Durante este tiempo, el controlador activo asume la responsabilidad total del control del sistema.

Si el controlador perdido se recupera y vuelve a estar operativo, el comparador lo reintegra en el sistema sin interrumpir la operación en curso, como si se hubiera simplemente reiniciado. El controlador recuperado se sincroniza con el estado actual del sistema y retoma su función en paralelo con su homólogo.

En cambio, si tras una serie de iteraciones el controlador finalmente no se recupera, y el sistema deja de ser redundante, se procederá a parar el proceso químico y llevarlo a un

estado seguro en el que aparcarlo, (descrito más adelante) debido a que el sistema ha perdido su característica de tolerancia a fallos.

La resolución de errores en el sistema se lleva a cabo de manera automática y transparente para el usuario. Cuando se detecta un error, el comparador evalúa la naturaleza del fallo y toma medidas correctivas. Estas medidas incluyen el reinicio de los controladores y la notificación al usuario a través de las pantallas OLED y los indicadores LED.

Si el comparador registra un error crítico, desconocido o el fallo de ambos controladores, detiene el proceso químico y lo lleva a un estado seguro, para lo que envía al tanque el comando de 'ABORT' que rápidamente cerrará la válvula de entrada, apagará el calentador, el enfriador y el agitador y, por último, abrirá la válvula de vaciado. La combinación de supervisión continua, respuesta rápida a fallos y recuperación automática asegura que el sistema pueda mantener su operación de manera robusta y eficiente, minimizando el tiempo de inactividad y maximizando la fiabilidad.

5

Implementación hardware del sistema

Como parte de este TFG se ha realizado una implementación hardware real del sistema de control redundante. En este capítulo se describen los componentes utilizados, incluyendo las placas Arduino Nano, pantallas OLED, LEDs y demás elementos electrónicos, así como su interconexión y montaje.

Además, se presentará la interfaz gráfica usada para las pruebas, programada en Python y desarrollada con el software PAGE (Python Automatic GUI Generator), que es una herramienta de desarrollo que facilita la creación de interfaces gráficas de usuario (GUI) tkinter para Python.

5.1. Descripción del sistema

El sistema está compuesto por tres placas Arduino Nano que trabajan conjuntamente para controlar un tanque de reacción, en este caso emulado en un ordenador portátil, pero diseñado para que pueda ser aplicado perfectamente a un tanque real. Dos de los Arduinos actúan como controladores gemelos, mientras el tercer Arduino actúa como comparador.

Los gemelos están encargados de ejecutar el mismo algoritmo de control, monitorizando y gestionando las variables (temperatura y volumen) del tanque de reacción. Cada uno de estos Arduinos está conectado al comparador mediante cuatro cables, utilizando el protocolo Custom SPI, desarrollado específicamente para este proyecto, para la transmisión serie de datos entre los controladores y el comparador.

El Arduino comparador juega un papel central en la arquitectura del sistema, recibiendo las señales de los dos controladores y comparándolas para detectar cualquier discrepancia.

Este Arduino está conectado al ordenador mediante un puerto USB, a través del cual se realiza la conexión con la simulación software del tanque. La interacción con el tanque emulado se realiza a través de mensajes en la terminal serie del ordenador, permitiendo monitorizarlo y controlarlo en tiempo real.

Cada uno de los componentes del sistema está equipado con una pantalla OLED de 1.3 pulgadas, que muestra su estado en tiempo real. Estas pantallas proporcionan información visual sobre las operaciones que están realizando sobre el tanque y el estado general del sistema, facilitando su supervisión y diagnóstico.

El comparador, además de la pantalla OLED, tiene un LED y un anillo de LEDs que se utilizan para indicar cuando se detecta una discrepancia entre las señales de los controladores gemelos. Estos indicadores visuales son necesarios para alertar sobre cualquier inconsistencia que pueda requerir atención inmediata.

Los dos controladores, están conectados también por dos líneas de comunicación adicionales para la sincronización. Además, uno de los Arduinos gemelos está conectado al comparador mediante una línea extra que se utiliza para indicar cuándo debe reiniciarse; tal como se ha descrito en la sección 4.2.3. (Por simplicidad, en la implementación hardware para este TFG solo se están generando fallos en uno de los dos controladores).

Cada controlador tiene también un LED configurado para indicar cuando se está enviando un comando al comparador. Estos LEDs son muy útiles para visualizar el proceso de sincronización entre los controladores que se explica en la sección 4.2.2.

Por último, se incluye un botón pulsador que sirve para iniciar el sistema y comenzar las operaciones con el tanque de reacción emulado. Al presionar este botón, los Arduinos, que comienzan en un estado de espera, empiezan a ejecutar sus algoritmos de control, interactuando con el tanque y entre ellos.

5.2. Componentes hardware

5.2.1. Arduino Nano

El Arduino Nano es un microcontrolador basado en el ATmega328P. Es pequeño, completo y compatible con la mayoría de los accesorios diseñados para Arduino y todas sus librerías. Las librerías de Arduino son colecciones de código C++ que permiten realizar tareas específicas

de manera más sencilla en el entorno Arduino, y facilitan la interacción con el hardware y la implementación de algoritmos complejos. La versión 'Nano' cuenta con catorce pines digitales de entrada o salida, ocho entradas analógicas, un botón de reset, LED de encendido y se puede montar en una protoboard.

Para el proyecto, se han utilizado tres placas Arduino Nano: dos de ellas actúan como controladores gemelos, ejecutando el mismo algoritmo de control para el tanque de reacción, mientras, la tercera unidad funciona como un comparador, recibiendo y verificando las salidas de los controladores gemelos para asegurar el correcto envío de los comandos a los actuadores del sistema.

5.2.2. Componentes electrónicos

Los componentes electrónicos utilizados incluyen protoboards, cableado, resistencias, LEDs, jumpers y un botón. Estos elementos son esenciales para la construcción y el montaje de los circuitos necesarios para el funcionamiento del sistema.

- Protoboards: Utilizados para el montaje temporal de circuitos electrónicos, permitiendo conexiones rápidas y modificaciones sencillas.
- Cableado y Jumpers: Empleados para establecer conexiones entre diferentes componentes y módulos en la protoboard.
- Resistencias: Utilizadas para limitar la corriente y proteger otros componentes electrónicos.
- LEDs: Implementados como indicadores visuales para mostrar el estado de diferentes partes del sistema.
- Botones: Usados para la interacción manual con el sistema, permitiendo, por ejemplo, reiniciar el sistema o introducir comandos específicos.

5.2.3. Pantallas LCD

Las pantallas OLED utilizadas son módulos de visualización de cuatro pines, con una resolución de 128x64 y un tamaño de 1.3 pulgadas. Siguen el protocolo de comunicación Inter-Integrated Circuit (I²C). Estas pantallas ofrecen una gran calidad de imagen y resultan muy útiles en proyectos Arduino.

En este proyecto, se han utilizado tres de estas pantallas OLED, cada una asignada a un Arduino. Cada una muestra el estado de su Arduino, proporcionando información en tiempo real sobre parámetros críticos como el paso de la ejecución, (si se está llenando o calentando el tanque e incluso si se está reiniciando), en el que se encuentra, lo que facilita la monitorización y diagnóstico del sistema.

5.2.4. Anillo LED

El anillo LED utilizado contiene ocho LEDs, cada uno de los cuales puede ser controlado individualmente para mostrar diferentes colores y patrones de luz. Son LEDs del tipo WS2812B.

Se utiliza como un indicador visual para alertar sobre el reinicio de alguno de los Arduinos debido a la detección de discrepancias en los comandos recibidos de los controladores. Cuando se detecta una inconsistencia y uno de los Arduinos se reinicia, el anillo LED muestra un patrón de luz específico para notificar a los operadores del sistema sobre esta situación.

5.3. Montaje del sistema

Como ilustra la Figura 19, el ensamblaje del sistema comenzó montando dos Arduino Nano en dos protoboards, conectándoles los cuatro cables necesarios para el protocolo Custom SPI y añadiendo un LED de estado. Una vez que se consiguió hacer funcionar la conexión SPI con dos Arduinos, se añadió el tercero, como muestra la Figura 20.

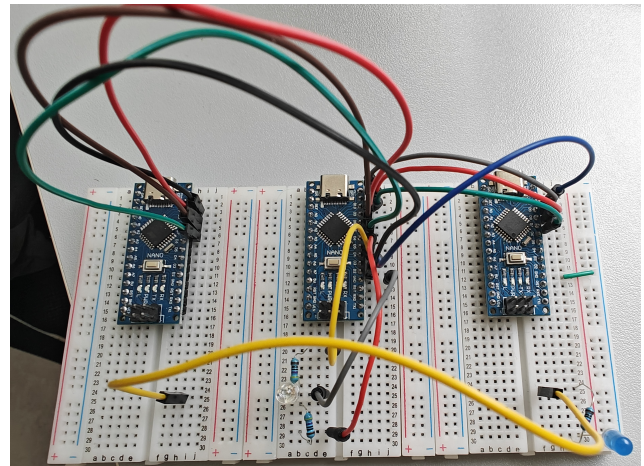
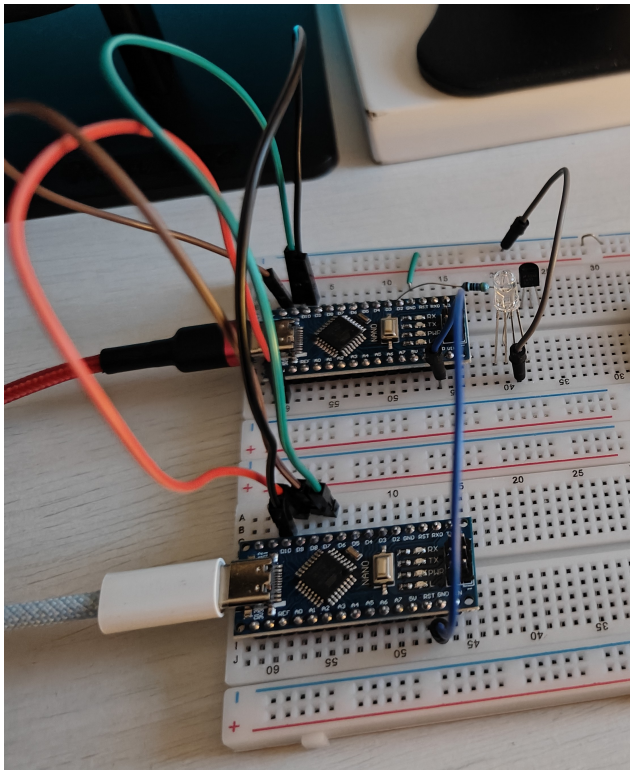


Figura 19: Conexión entre dos Arduino Nano Figura 20: Conexión entre tres Arduino Nano

A continuación, se añadió al sistema el botón para iniciar su operación y algunos LEDs para el control del mismo, (Figura 21). Más tarde se reorganizó el cableado para hacerlo más estético y, sobre todo, para hacer más fácil, seguir añadiendo componentes (Figura 22).

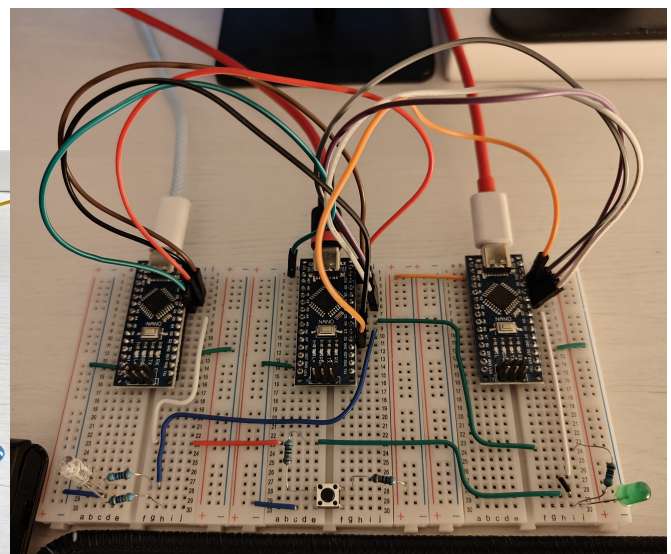
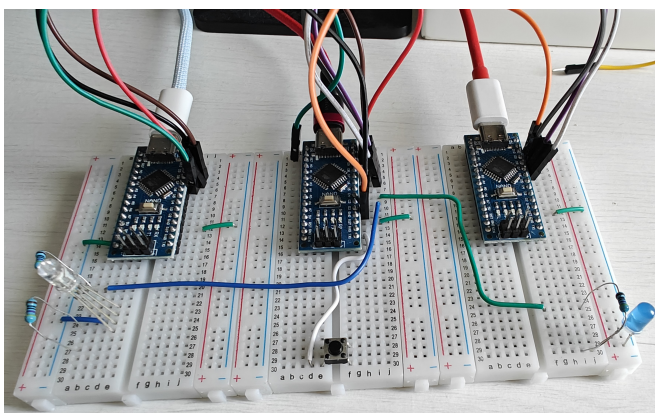


Figura 21: Se añade un botón y LEDs Figura 22: Reorganización de los componentes

Llegados a este punto se comprobó que el modelo usado de protoboard no tenía el tamaño suficiente para albergar al sistema, por lo que se procedió (Figura 23), al traslado de los componentes a protoboards más grandes, tras lo cual comenzaron las pruebas con las pantallas LCD OLED; la Figura 24 muestra otro entorno más sencillo utilizado para las pruebas con la pantalla antes de su integración en el sistema.

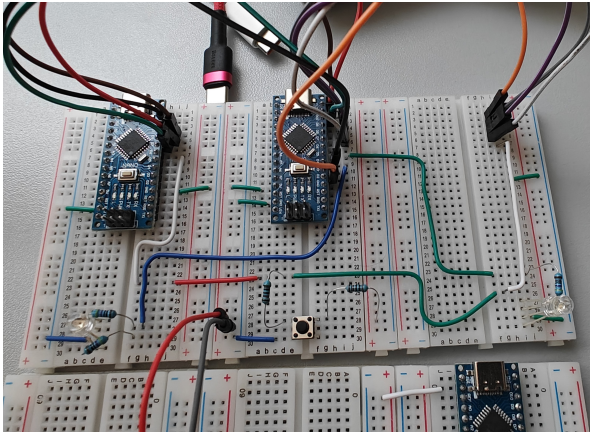


Figura 23: Traslado del sistema a protoboards de mayor tamaño

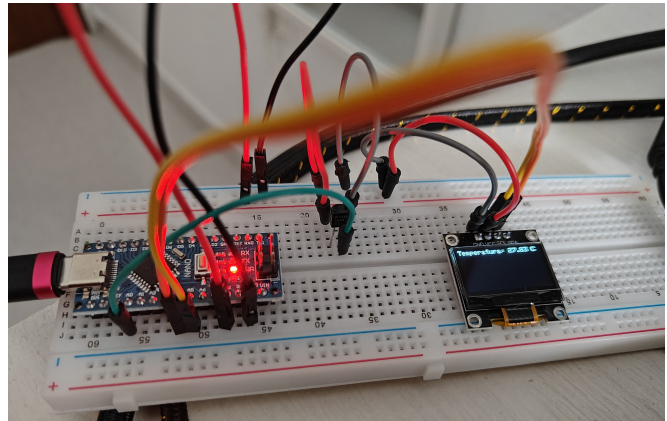


Figura 24: Prueba de la pantalla LCD con un Arduino

La Figura 25 muestra el resultado de la integración de la pantalla con el Arduino comparador. Dado el buen resultado de la pantalla en el sistema, se procedió a integrar otras dos más, cada una para uno de los controladores gemelos, (Figura 26).

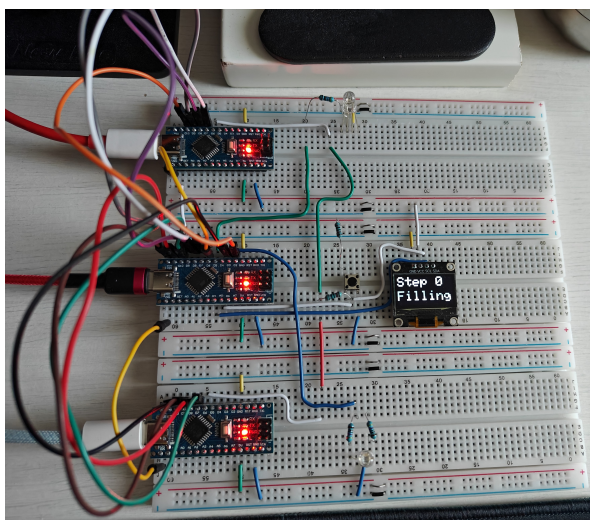


Figura 25: Se añade al sistema la pantalla LCD

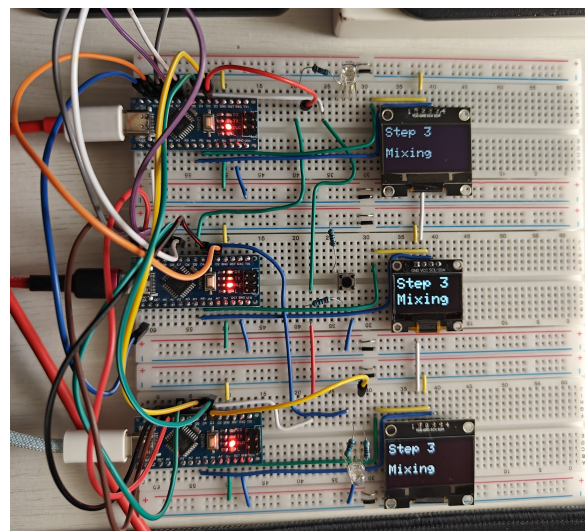


Figura 26: Se añaden al sistema más pantallas

Llegados a este punto se cambió la pantalla del comparador por una del mismo tamaño que las de los gemelos, y se añadió, también, un LED para indicar discrepancia por parte de alguno de los controladores y que indica también que se procede al reinicio del mismo, (Figura 27).

Finalmente, se cambió el LED de discrepancia por un anillo de LEDs, se añadió un LED de reinicio a cada controlador y se realizó la última reorganización de los componentes del sistema, (Figura 28).

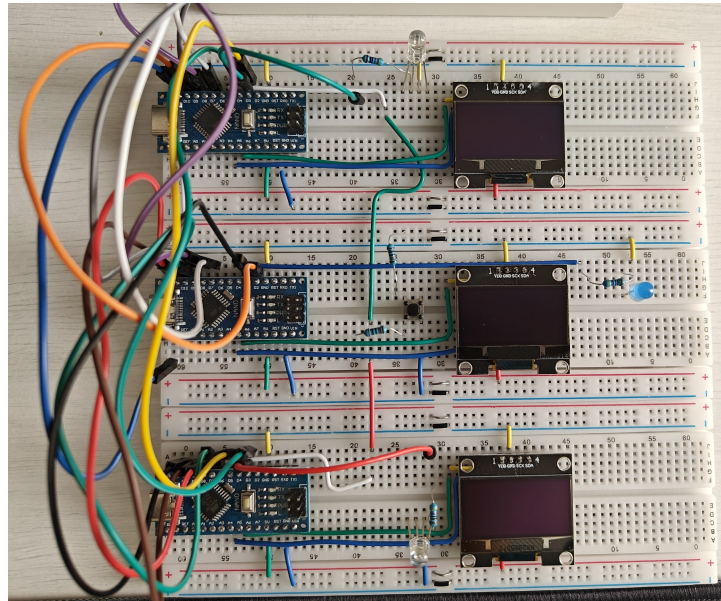


Figura 27: Nueva pantalla para el comparador y LED de discrepancia

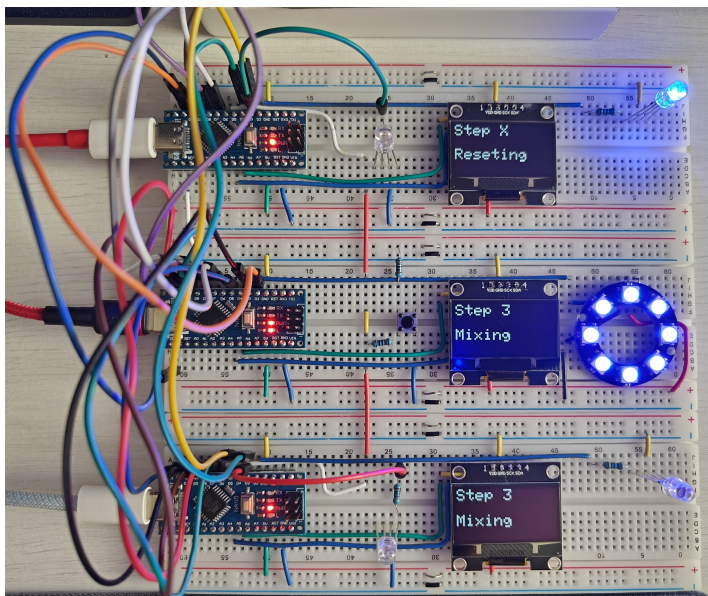


Figura 28: Resultado final del sistema

5.3.1. Diagrama y esquemático

El diagrama resultante para la implementación hardware del sistema es el siguiente:

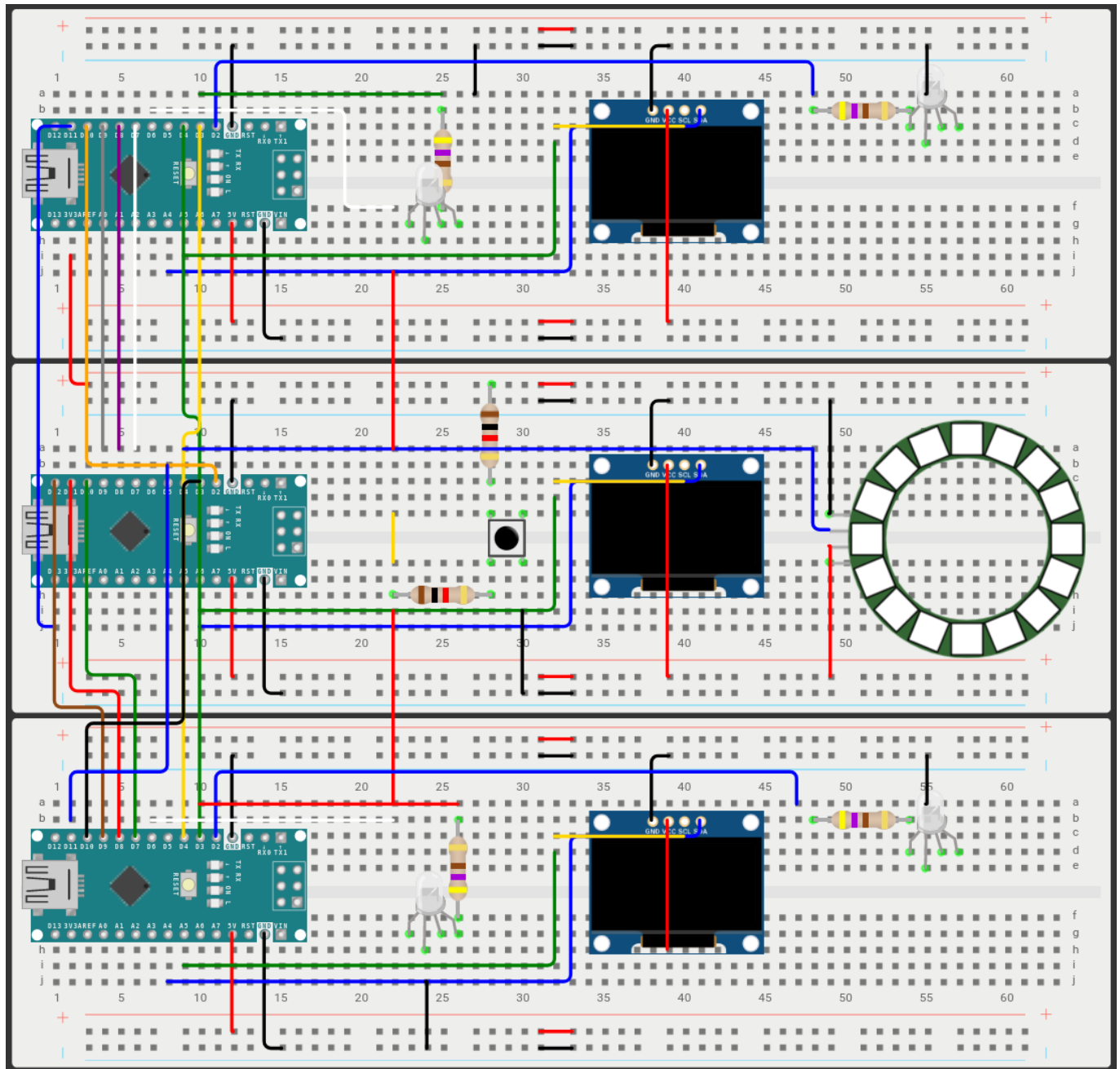


Figura 29: Diagrama final del sistema

El esquemático con el pinout y las conexiones del sistema es el siguiente:

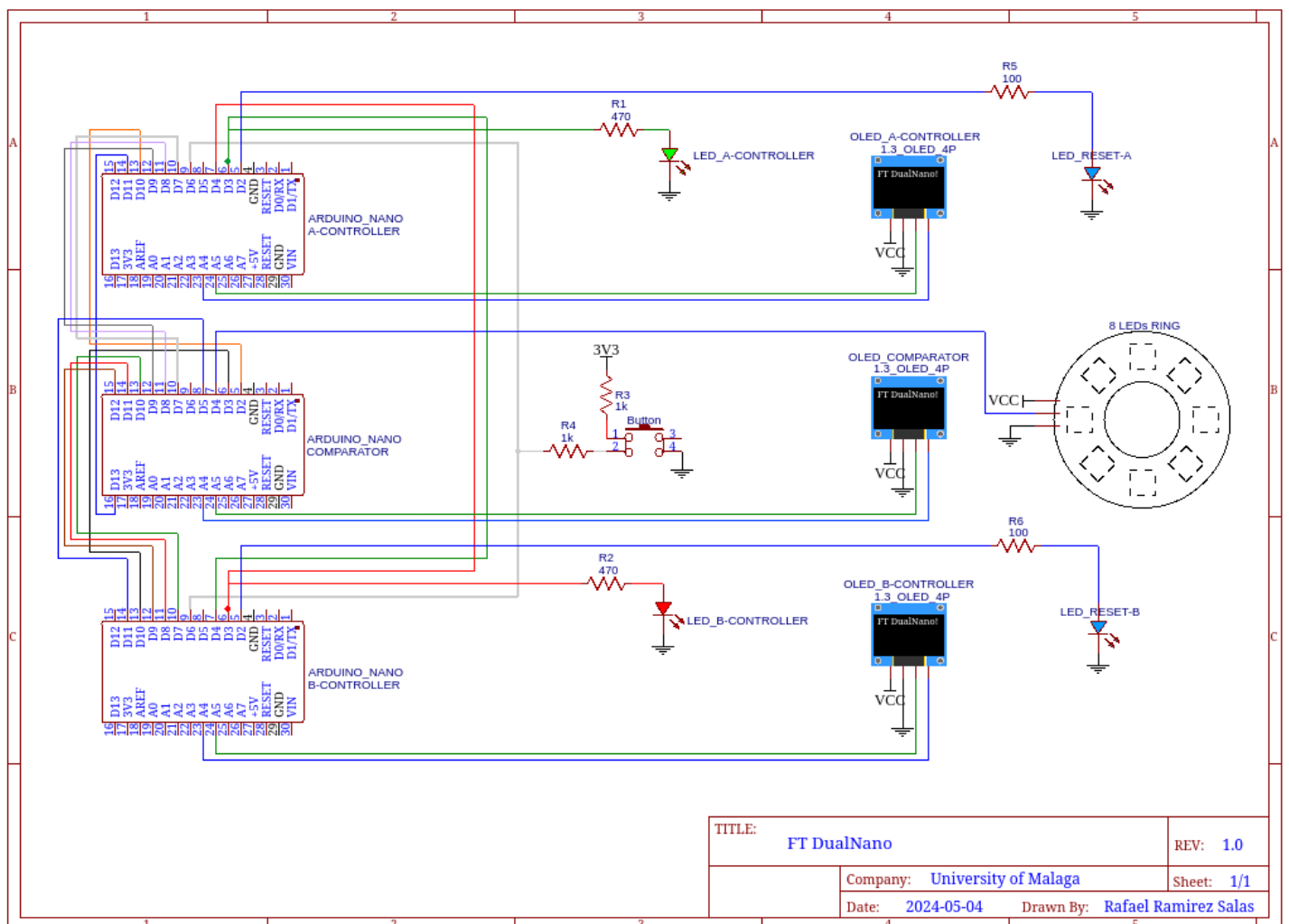


Figura 30: Esquemático del sistema

5.4. Interfaz Gráfica de usuario

Para poder hacer de forma práctica el seguimiento de la operación del sistema, ha sido necesario programar en Python una aplicación que proporciona una visualización del estado del sistema y de sus mensajes de control. La Figura 31 muestra el diseño de la interfaz gráfica de la aplicación.

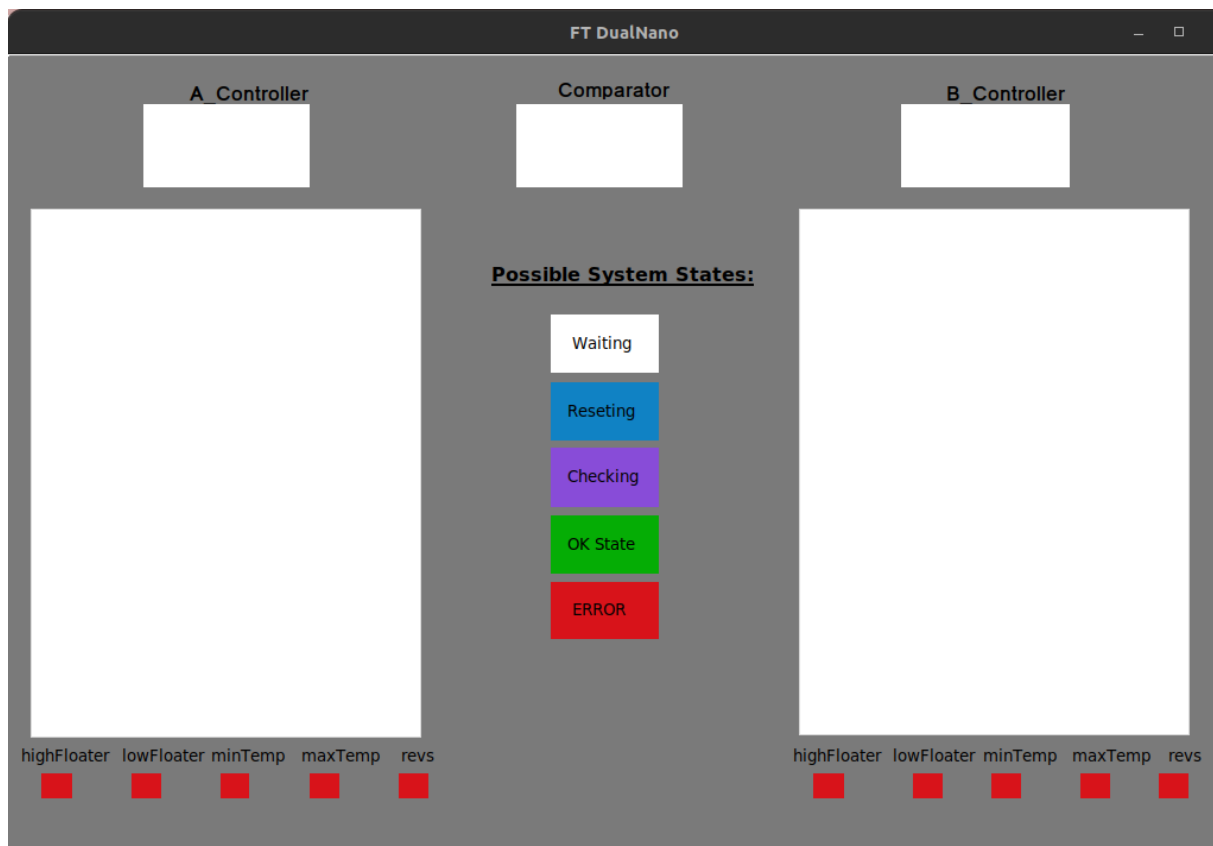


Figura 31: Interfaz gráfica con el sistema apagado

Como se aprecia en la Figura, cada componente del sistema tiene asignado un recuadro, que dependiendo del estado adoptará un color u otro. Concretamente, el color blanco significa que el dispositivo está esperando la pulsación del botón en la protoboard para iniciar el proceso químico en el tanque.

Por su parte, el color azul indica que el controlador se está reseteando, mientras que si adopta un color morado el algoritmo del controlador está en el paso intermedio de chequeo, comprobando que los valores son los adecuados antes de pasar al siguiente step. El color verde es el que más se repite, ya que indica que el controlador o el comparador está trabajando

correctamente, en contraposición al color rojo que indica un error crítico que ha resultado en el cese del proceso.

En los cuadros de texto se capturan y muestran los mensajes procedentes de cada controlador, para saber en cada momento el estado en el que se encuentran, el comando enviado y la temperatura y el volumen del tanque. También se muestra en ellos cuándo el comparador ha enviado un comando de reinicio, o si ocurre algún fallo crítico que haga detener la ejecución y trasladar al sistema al estado de error.

Por otro lado, el resto de datos del tanque son los cinco flags de 'true' o 'false' que indican si el tanque ha llegado al mínimo o al máximo de líquido, si el contenido ha alcanzado la mínima o máxima temperatura y si el motor está o no encendido; todos ellos están representados en pequeños cuadros, que oscilan entre color rojo o verde dependiendo de si su valor es falso o verdadero, y están localizados debajo del cuadro de texto. Cada controlador tiene sus propios flags que son modificados de acuerdo a los valores que haya recibido su controlador asociado.

Debido a un fallo en algún sensor, un reinicio accidental o un apagado momentáneo, se puede producir una discrepancia en los comandos enviados al comparador. Cuando esto ocurre, un comando de reset es enviado al controlador que ha emitido el comando erróneo. En el ejemplo ilustrado en la Figura 32, el Controlador B ha sufrido el fallo de un sensor y, recalcado en color azul, se lee cómo tras el fallo se le ha enviado un comando de Reset que le indica que debe reiniciarse e incorporarse al 'STEP1' de la ejecución, tras lo cual el proceso continúa correctamente hasta que vuelve a producirse otro fallo en un sensor, que se corrige de la misma manera.

En situaciones donde el sistema deja de ser redundante, pierda ambos controladores, o el contenido del tanque se vuelve inestable o ocurre alguna anomalía que produce un fallo crítico, el sistema entra en un estado de ERROR. En este estado se detiene toda actividad en el tanque y, por razones de seguridad, el error se propaga a todos los componentes del sistema. Cuando ocurre un error crítico, el sistema se detiene y pone el tanque en modo seguro, vaciando su contenido, cerrando la válvula de entrada, apagando el calefactor y el enfriador, y vaciando el tanque. El sistema permanece en espera de intervención humana antes de iniciar un nuevo proceso, como se muestra en la Figura 33.

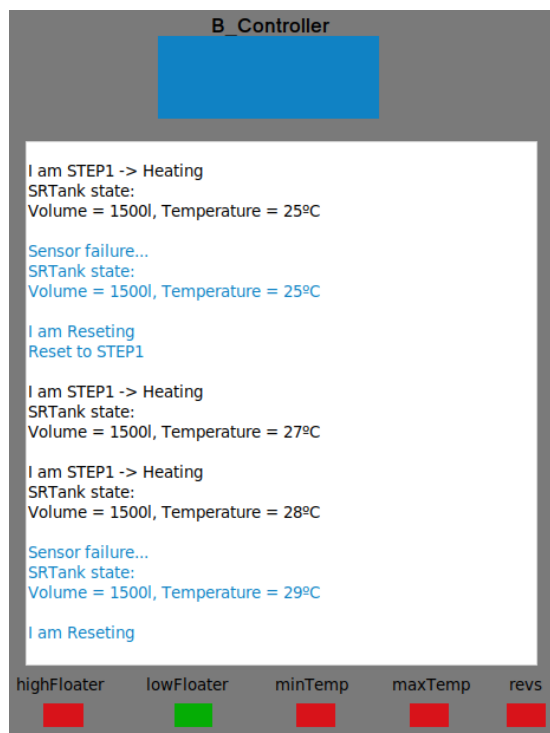


Figura 32: Acción de reset

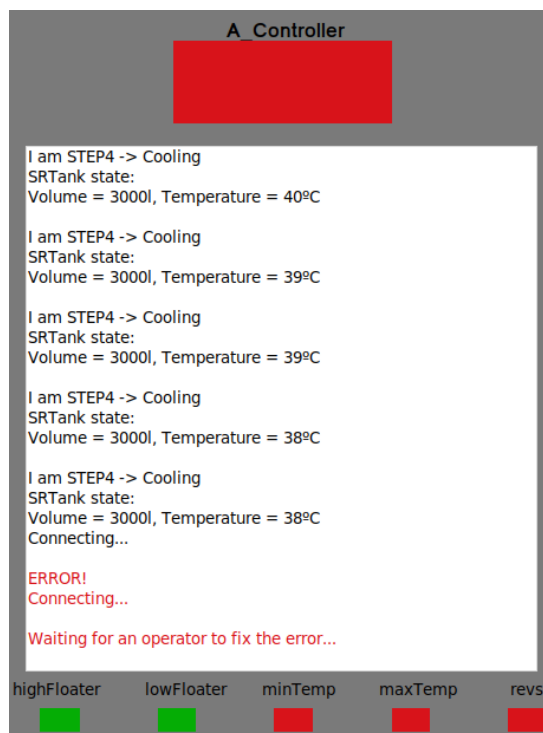


Figura 33: Acción de ERROR

5.4.1. Implementación

Para la implementación de la interfaz gráfica de usuario se ha utilizado el software PAGE (Python Automatic GUI Generator), una herramienta open source que facilita el diseño de interfaces gráficas utilizando Python y Tkinter. Esta herramienta permite crear interfaces de manera cómoda y sencilla mediante una dinámica de arrastrar y soltar, generando el esqueleto de código en Python necesario para dar funcionalidad a todos los elementos.

Con PAGE, se ha diseñado una pantalla intuitiva y funcional para controlar el sistema. El diseño incluye elementos visuales como cuadros de colores, cuadros de texto y alertas que permiten a los usuarios interactuar con el sistema de manera visual. El back-end de la aplicación está programado en Python, lo que permite una integración fluida con los Arduinos y los protocolos de comunicación utilizados.

Como era la primera vez que programaba en Python, me ha sido necesario aprender a utilizar dicho lenguaje, incluyendo las librerías y componentes necesarios para que la interfaz gráfica creada con PAGE funcionara. Es importante destacar en este punto que, dato que el lenguaje Python no está incluido en los contenidos de la titulación, he debido aprender a usar

el lenguaje y sus librerías partiendo totalmente de cero. Gracias a esto, se ha creado una herramienta que ayuda a una mejor visualización del sistema y muestra lo importante que es seguir aprendiendo y adaptarse a las nuevas tecnologías cuando se trabaja en nuevos proyectos.

5.5. Pruebas del sistema

Para asegurar el correcto funcionamiento del sistema se han realizado varias pruebas, incluyendo la simulación completa del proceso químico, la simulación de errores y la manipulación manual de los controladores para poner a prueba la capacidad del sistema de manejar y recuperarse de fallos.

5.5.1. Proceso químico completo

Como se define en la sección 2.2.1, el proceso químico simulado es conocido como ‘saponificación’. Para comprobar el correcto desempeño del sistema, se recrea la saponificación siguiendo una serie de pasos. Estos pasos aparecen acompañados de una explicación y de la representación de la evolución del tanque, junto a los mensajes mostrados por las pantallas OLED.

Los ‘steps’ que se llevan a cabo durante el proceso químico son los aquí listados:

1. Abrir la válvula de llenado del tanque con una solución ya mezclada de agua con sosa cáustica hasta alcanzar los 1500 litros: El proceso comienza con la apertura de la válvula de llenado del tanque. Una solución de agua y sosa cáustica (NaOH) se introduce en el tanque hasta alcanzar un volumen de 1500 litros, (Figuras 34 y 35); esta solución básica es necesaria para iniciar la reacción de saponificación.

NOTA: Cada vez que se lleva a cabo un paso, se realiza una comprobación intermedia antes de pasar al siguiente, (Figuras 36 y 37). Estos chequeos serán profundizados en la sección 5.5.1 y solo se llevan a cabo por los controladores.